

중요 어휘

1. **범용(汎用)**: 특정 분야에 한정되지 않고 여러 방면에 널리 쓰임.

동의어: 다용도(多用途)

반의어: 전용(專用)

예문: 이 소프트웨어는 **범용** 도구로서 다양한 업무에 활용할 수 있다.

2. **종속성(從屬性)**: 어떤 것이 다른 것에 매여 독립적으로 존재하거나 작동하지 못하는 성질.

동의어: 의존성(依存性)

반의어: 독립성(獨立性)

예문: 이 공정은 앞 단계와의 **종속성**이 커서 순서를 바꾸기 어렵다.

3. **조율(調律)하다**: 서로 다른 요소들을 알맞게 맞추어 조화롭게 다듬다.

동의어: 조정(調整)하다, 조절(調節)하다

예문: 팀장은 각 부서의 의견을 **조율하여** 최종 일정을 확정했다.

4. **집적(集積)하다**: 여러 가지를 한곳에 모아 쌓다.

동의어: 축적(蓄積)하다, 집약(集約)하다

반의어: 분산(分散)하다

예문: 반도체 기술의 발전으로 하나의 칩에 수십억 개의 트랜지스터를 **집적**할 수 있게 되었다.

5. **역전파(逆傳播)**: 출력에서 발생한 오차를 역방향으로 전달하여 각 단계의 가중치를 수정하는 과정.

예문: 인공 신경망은 **역전파** 알고리즘을 통해 학습 정확도를 높여 나간다.

6. **유체 역학(流體力學)**: 액체나 기체 등 유체의 운동과 그에 작용하는 힘을 연구하는 학문 분야.

예문: 항공기 날개의 설계에는 **유체 역학**의 원리가 핵심적으로 적용된다.

7. **디코딩(decoding)**: 부호화된 데이터를 원래의 형태나 의미로 해독하는 과정.

반의어: 인코딩(encoding)

예문: 수신된 신호는 **디코딩** 과정을 거쳐 사람이 이해할 수 있는 정보로 변환된다.

8. **스케줄러(scheduler)**: 여러 작업이나 자원의 실행 순서와 배분을 결정하여 관리하는 장치 또는 프로그램.

예문: 운영 체제의 **스케줄러**가 각 프로세스에 적절한 실행 시간을 할당한다.

9. **계층화(階層化)하다**: 등급이나 단계에 따라 위아래의 층을 나누어 체계적으로 구성하다.

동의어: 단계화(段階化)하다

반의어: 평준화(平準化)하다

예문: 회사는 보안 등급에 따라 정보 접근 권한을 **계층화**하여 관리했다.

10. **일괄(一括)**: 여러 가지를 하나로 묶어 한꺼번에 처리함.

동의어: 일제(一齊), 일시(一時)

반의어: 개별(個別)

예문: 월말에 밀린 서류를 **일괄** 처리하여 업무를 마무리했다.

11. **분기문(分岐文)**: 주어진 조건에 따라 프로그램의 실행 경로를 둘 이상으로 나누는 명령 구조.

예문: 프로그래밍에서 if-else 구문은 대표적인 **분기문**에 해당한다.

12. **삽입(插入)하다**: 이미 있는 것 사이에 다른 것을 끼워 넣다.

동의어: 삽입(插入)하다, 추가(追加)하다

반의어: 삭제(削除)하다

예문: 보고서 중간에 참고 도표를 **삽입**하여 독자의 이해를 도왔다.

13. **운용(運用)하다**: 사물이나 제도, 조직 따위를 움직이고 부려서 쓰다.

동의어: 활용(活用)하다, 구사(驅使)하다

예문: 학교는 한정된 예산을 효율적으로 **운용**하여 다양한 프로그램을 진행했다.

14. **유기적(有機的)**: 각 부분이 서로 긴밀하게 연결되어 전체로서 통일된 기능을 발휘하는.

동의어: 체계적(體系的)

반의어: 단편적(斷片的)

예문: 인체의 각 기관은 **유기적**으로 연결되어 하나의 생명 활동을 유지한다.

[1~8] 다음 글을 읽고 물음에 답하십시오.

최근까지 일반적인 컴퓨터 환경에서는 운영 체제를 제어하거나 사용자 인터페이스를 처리하고, 문서를 작성하거나 웹을 탐색하는 등 다양한 범용 작업이 주요 연산 대상이었다. 이러한 작업은 명령어의 종류가 많고 그 순서가 수시로 바뀌며, 연산 간 종속성이 크다는 특징이 있다. 컴퓨터의 연산은 주로 여러 개의 코어로 구성된 중앙 처리 장치(CPU)에서 수행된다. 코어는 컴퓨터의 두뇌 역할을 하는 것으로 명령어를 해석하고 각종 연산을 실행한다. CPU는 여러 개의 고성능 코어로 구성되어 복잡한 제어 흐름을 정밀하게 조율할 수 있도록 설계되었다. 그러나 인공 지능 학습, 고해상도 영상 처리, 과학 기술 관련 계산 등 대규모 데이터를 대상으로 동일한 연산을 반복 수행해야 하는 작업이 증가하면서 기존의 연산 환경에 변화가 생겼다. ㉠[CPU는 코어 수에 한계가 있고 개별 코어가 복잡한 제어 흐름에 맞추어 다양한 연산을 처리하도록 설계되어 있기 때문에 동일한 연산을 대량으로 반복 수행해야 하는 현대의 병렬 연산 환경을 효율적으로 처리하기 어렵다.] 이러한 상황에서 컴퓨터의 새로운 중심 장치로 떠오른 것이 그래픽 처리 장치(GPU)이다.

GPU는 컴퓨터 내에서 그래픽 처리를 담당하는 장치로, 수많은 픽셀에 동일한 연산이 반복적으로 이루어지는 그래픽 연산의 특성상 다수의 코어가 병렬로 작동하는 구조로 이루어져 있다. GPU는 수백 개에서 수천 개에 이르는 ㉡[단순 연산 전용 코어]를 하나의 칩에 집적한 상태로, 많은 일꾼이 한 가지 작업을 동시에 수행하는 공장 같다. 이는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있으며, GPU의 코어는 제어 기능을 최소화하고, 동일한 연산을 빠르게 수행하도록 설계되어 있다. 이러한 구조를 바탕으로 GPU는 원래의 목적인 그래픽 처리 외에도 다양한 분야에서 활용되고 있다. 인공 지능 학습에서는 행렬 곱셈과 역전파 계산 등 동일한 연산의 반복이 이루어지며, 영상 처리나 고해상도 그래픽 구현에서는 수많은 픽셀 단위의 연산이 병렬로 적용된다. 또한 기상 예측, 유전자 배열 분석, 유체 역학 시뮬레이션 등에서도 대규모 수치 연산을 빠르게 처리해야 하기 때문에, 병렬 구조에 기반한 연산 장치가 필요하다.

GPU 내부에는 수많은 연산 코어가 집적되어 있고 이들은 일정 단위로 묶여 연산 유닛을 형성한다. 연산 유닛은 여러 개의 스레드를 병렬로 실행할 수 있는 독립적인 실행 공간으로, 공장 내에서 같은 작업을 수행하는 작업반에 비유할 수 있다. 스레드는 연산을 수행하는 최소 단위로, 하나의 스레드는 하나의 명령어를 실행한다. GPU는 여러 스레드를 하나의 스레드 블록으로 묶어서 관리하는데, 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행하게 된다. 명령어가 GPU에 전달되면 디코딩 과정을 통해 해석되며, 스케줄러는 이 명령어에 따라 스레드 블록을 각 연산 유닛에 분배하고 병렬 처리가 이루어지도록 제어한다. 만약 연산량이 균등하게 분배되지 않으면 일부 연산 유닛은 다른 유닛의 연산이 끝날 때까지 대기 상태에 놓이게 되어 전체 GPU의 자원이 비효율적으로 사용되는 문제가 발생할 수 있다. 분배가 끝나면 스레드 블록은 병렬로 연산을 실행하고, 실행한 결과는 GPU 내의 메모리에 저장된다.

GPU는 연산 속도를 높이기 위해 계층화된 메모리 구조를 갖추고 있다. 메모리는 접근 속도, 용량, 사용 범위 등에 따라 전역 메모리, 공유 메모리, 레지스터로 나뉜다. 전역 메모리는 모든 스레드가 접근할 수 있는 대용량 저장

공간으로, 다음 단계의 연산이나 외부와의 공유를 위해 연산 실행 결과를 저장한다. 다만 접근 속도가 느리다는 단점이 있기 때문에 연산 중에 사용하기에는 적합하지 않다. 따라서 연산 중에는 개별 스레드만 사용할 수 있는 저장소인 레지스터나 같은 스레드 블록 내의 스레드들이 공동으로 사용하는 공유 메모리를 활용하여 데이터를 저장하고 처리하는 방식으로 성능을 최적화한다. 레지스터는 용량은 작지만 접근이 매우 빠르고, 공유 메모리는 전역 메모리보다 빠르면서도 여러 스레드 간의 정보 공유가 가능하다는 장점이 있다. ㉢[다만, 같은 스레드 블록 내의 여러 스레드가 동시에 같은 메모리 공간에 접근할 경우, 충돌이 발생해 병렬성이 저하되어 연산이 지연되거나 처리 속도가 저하되는 현상이 나타날 수 있다.] 따라서 메모리의 효율적 분배와 접근 방식의 조정은 GPU 연산 성능에 중요한 영향을 미친다.

[B]

한편, GPU는 동일한 명령어를 다수의 데이터에 일괄 적용하는 방식에 최적화되어 있지만, ㉣[분기문]이나 조건 판단이 자주 등장하는 연산에서는 병렬 처리의 효율이 낮아질 수 있다. 조건 판단은 특정 수치나 상태에 따라 연산의 흐름을 다르게 결정하는 과정이며, 분기문은 이러한 조건 판단에 따라 실행 경로를 분리하거나 연결하는 명령 구조를 의미한다. 예컨대, '어떤 값이 일정 기준보다 크면 A 연산을, 그렇지 않으면 B 연산을 수행하라.'는 명령이 있을 경우, 스레드마다 실행해야 할 작업이 달라진다. ㉤[GPU는 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있기 때문에, 이러한 분기문이 삽입되면 일부 스레드는 다른 스레드의 작업이 끝날 때까지 대기하게 되고, 그 결과 병렬성이 손실되는 현상이 발생한다.] 따라서 GPU는 단순하고 반복적인 연산에는 매우 효과적이지만, 연산 경로가 자주 갈라지거나 복잡한 조건 판단이 요구되는 작업에서는 성능이 저하될 수 있다.

CPU와 GPU의 장점을 고려하여, 최근의 컴퓨팅 시스템은 두 장치의 기능을 나누어 운용하는 구조로 발전하고 있다. 각 장치의 강점을 바탕으로 전체 시스템을 구성하여 연산 효율을 높이는 것이다. ㉥[연산 작업의 성격과 목적]에 따라 CPU와 GPU 간 연산 흐름을 유기적으로 조정할 수 있도록 하드웨어 및 소프트웨어 설계 방식이 변화하고, 두 연산 장치의 성능을 최대한으로 끌어내면서도 효과적인 활용을 위한 프로그래밍이 이루어지고 있다. 이렇게 현대의 컴퓨팅 시스템은 두 연산 장치가 각자의 구조적 강점을 최대한 발휘할 수 있도록 하여 더욱 복잡하고 대규모화된 연산 환경에 대응하고 있다.

* 픽셀: 주소화될 수 있는 화면의 가장 작은 단위. 작은 점의 행과 열로 이루어져 있는 화면의 작은 점 각각을 이르는 말.

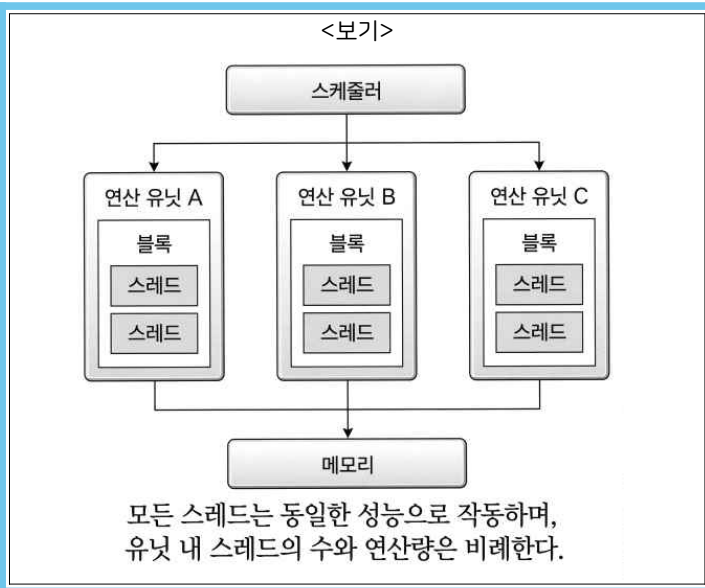
1. 밑글의 서술 방식에 대한 설명으로 가장 적절한 것은?

- ① 두 장치의 내부 회로 구성을 단계적으로 제시한 뒤, 각 단계에서의 전력 소비량 차이를 비교하고 있다.
- ② 연산 환경의 변화를 배경으로 제시하고, 이에 대응하는 장치의 구조와 작동 원리를 설명한 뒤 한계와 보완 방향을 서술하고 있다.
- ③ 동일한 실험 조건에서 두 장치의 처리 속도를 측정한 결과를 수치로 제시하며, 정량적 비교를 통해 우열을 판정하고 있다.
- ④ 여러 세대의 그래픽 처리 장치를 시간순으로 나열하며, 기술 발전에 따른 코어 수의 증가 추이를 통시적으로 조망하고 있다.
- ⑤ 이론과 실제 적용 사례를 번갈아 제시하면서 이론적 예측과 실제 성능 사이의 괴리를 분석하고 원인을 규명하고 있다.

2. 밑줄에 대한 이해로 적절하지 않은 것은?

- ① GPU의 코어는 CPU의 코어에 비해 제어 기능이 축소되어 있으나, 동일한 연산을 반복 처리하는 데에는 더 효율적이다.
- ② CPU는 명령어의 종류가 많고 순서가 수시로 바뀌며 연산 간 종속성이 큰 작업에 적합하도록 설계되었다.
- ③ GPU 내부의 스레드 블록이 연산 유닛에 분배된 후 병렬로 연산을 실행하면, 그 결과는 GPU 내의 메모리에 저장된다.
- ④ 공유 메모리는 레지스터보다 접근 속도가 빠르면서도 같은 스레드 블록 내의 여러 스레드가 함께 사용할 수 있다는 장점이 있다.
- ⑤ 최근의 컴퓨팅 시스템은 CPU와 GPU의 구조적 강점을 모두 활용할 수 있도록 하드웨어 및 소프트웨어 설계 방식이 변화하고 있다.

3. <보기>는 GPU의 구조를 나타낸 것이다. 밑글과 [A], [B]를 바탕으로 <보기>를 이해한 내용으로 적절하지 않은 것은?



- ① 유닛 A, B, C에 각각 하나의 스레드 블록이 배치되어 있으므로, 스케줄러가 스레드 블록을 균등하게 분배한 결과에 해당한다.
- ② 유닛 A의 블록에 속한 두 스레드는 기본적으로 동일한 명령어를 실행하며, 유닛 B나 유닛 C의 블록에 속한 스레드들도 마찬가지이다.
- ③ 연산이 완료된 후 각 유닛의 스레드 블록이 산출한 결과는 하단의 메모리, 즉 전역 메모리에 저장되어 다음 단계의 연산이나 외부와의 공유에 활용된다.
- ④ 유닛 A의 블록 내 두 스레드가 동시에 동일한 공유 메모리 공간에 접근하려 할 경우, 충돌이 발생하여 해당 유닛의 연산 처리 속도가 저하될 수 있다.
- ⑤ 유닛 A의 블록 내 두 스레드가 연산을 수행하는 과정에서 중간 결과를 하단의 메모리에 저장하여 처리하면, 모든 스레드가 접근할 수 있으므로 성능 최적화에 유리하다.

4. ㉔의 이유로 가장 적절한 것은?

- ① GPU의 스레드 블록이 연산 유닛에 균등하게 분배되지 않아 일부 유닛이 유휴 상태에 놓이기 때문에
- ② 전역 메모리의 접근 속도가 레지스터나 공유 메모리에 비해 현저히 느려서 대규모 데이터 처리 시 병목이 발생하기 때문에
- ③ CPU의 코어가 복잡한 제어 흐름을 정밀하게 조율하는 구조로 되어 있어, 동일 연산의 대량 반복에서는 코어당 처리량이 GPU보다 낮기 때문에
- ④ GPU의 코어 수가 수백에서 수천 개에 이르더라도, 각 코어의 개별 연산 처리 성능이 CPU의 고성능 코어보다 떨어지기 때문에
- ⑤ CPU의 개별 코어는 다양한 명령어의 해석과 복잡한 제어 흐름의 관리를 위해 설계되어 제어 기능에 상당한 자원이 할당되므로, 제어 기능을 최소화하고 동일 연산의 고속 반복에 특화된 코어를 대량으로 필요로 하는 병렬 연산 환경에는 구조적으로 부적합하기 때문이다.

5. ㉑과 ㉒에 대한 이해로 가장 적절한 것은?

- ① ㉑의 수가 많을수록 분기문이 삽입된 연산에서의 병렬성 손실 문제가 자연스럽게 해소된다.
- ② ㉑은 제어 기능을 최소화한 코어이므로, ㉒이 자주 등장하는 연산에서도 CPU의 고성능 코어와 동등한 수준의 효율을 발휘한다.
- ③ ㉒이 삽입되면 스레드마다 실행할 작업이 달라지므로, ㉑이 동일한 명령어를 동시에 수행하는 GPU의 설계 원리와 충돌이 발생한다.
- ④ ㉑의 수를 줄이고 각 코어의 제어 기능을 강화하면, ㉒이 자주 등장하는 연산에서의 병렬성 손실을 방지하면서도 GPU의 연산 총량을 유지할 수 있다.
- ⑤ ㉒이 없는 연산에서는 ㉑이 순차적으로 명령어를 처리하고, ㉒이 있는 연산에서는 ㉑이 병렬적으로 명령어를 처리한다.

6. ㉔를 분석한 내용으로 가장 적절한 것은?

- ① ㉔는 레지스터와 공유 메모리를 활용한 성능 최적화 전략의 한계를 환기하는 맥락에서, 메모리 충돌로 인해 전역 메모리의 접근 속도 문제까지 심화된다는 점을 부각하는 기능을 한다.
- ② ㉔는 레지스터와 공유 메모리를 활용한 성능 최적화 전략의 한계를 환기하는 맥락에서, 메모리의 효율적 분배와 접근 방식 조정이 연산 성능에 중요하다는 결론을 뒷받침하는 기능을 한다.
- ③ ㉔는 공유 메모리의 장점이 서술된 직후에 위치하여, 충돌 상황에서는 공유 메모리의 속도 이점 자체가 소멸됨을 지적함으로써 공유 메모리 도입의 근거를 약화하는 기능을 한다.
- ④ ㉔는 공유 메모리의 장점이 서술된 직후에 위치하여, 해당 장점에 불구하고 발생할 수 있는 문제를 보충함으로써 전역 메모리가 연산 중에도 불가결함을 입증하는 기능을 한다.
- ⑤ ㉔는 GPU의 병렬 처리 구조에 내재한 근본적 결함을 드러내어, CPU가 메모리 관리 측면에서 GPU보다 우월하다는 결론을 도출하기 위한 전제로 기능한다.

7. ㉔를 바탕으로 ㉕의 의미를 추론한 내용으로 가장 적절한 것은?

- ① ㉔에서 드러난 GPU의 한계는 모든 연산을 GPU에 집중시키는 설계가 최적임을 보여 주며, ㉕은 GPU에 최적화된 작업만을 선별하여 할당하는 기준을 가리킨다.
- ② ㉔에서 분기문이 삽입될 때 GPU의 병렬성이 손실된다는 점은, ㉕이 단순 반복 연산과 복잡한 조건 판단 연산을 구분하여 각각에 적합한 장치에 배분하는 기준임을 시사한다.
- ③ ㉔에서 일부 스레드가 대기하게 된다는 점은 GPU의 스케줄러가 비효율적으로 작동함을 의미하며, ㉕은 스케줄러의 분배 알고리즘을 개선하기 위한 소프트웨어적 기준을 가리킨다.
- ④ ㉔에서 병렬성이 손실되는 현상은 GPU의 메모리 계층 구조에서 비롯되며, ㉕은 메모리 접근 방식에 따라 CPU와 GPU의 역할을 재편하는 하드웨어 설계 기준을 가리킨다.
- ⑤ ㉔에서 모든 스레드가 동일한 명령어를 수행해야 하는 제약은 CPU에도 동일하게 적용되므로, ㉕은 CPU와 GPU가 동일한 연산을 분담하되 처리 속도에 따라 비율을 조정하는 기준을 가리킨다.

8. 밑글을 바탕으로 <보기>를 이해한 내용으로 가장 적절한 것은?

<보기>

한 의료 영상 분석 시스템은 대량의 CT 촬영 이미지를 분석하여 종양 의심 부위를 자동으로 표시하는 작업을 수행한다. 이 시스템은 먼저 환자별 촬영 이미지의 메타데이터를 분석하여 촬영 조건과 환자 정보를 정리하고, 각 이미지의 해상도와 밝기를 표준화하는 전처리 과정을 거친다. 이후 표준화된 수천 장의 이미지에 대해 동일한 필터를 적용하여 종양 의심 부위의 윤곽을 추출하는 연산을 병렬로 수행한다. 그런데 윤곽 추출 과정에서 특정 이미지의 밝기 값이 기준치를 벗어나면 보정 연산을 추가로 수행하고, 기준치 이내이면 보정 없이 다음 단계로 넘어가는 조건 분기가 포함되어 있다. 최종적으로 분석 결과를 종합하여 환자별 보고서를 생성하는 단계에서는, 환자 이력과 촬영 기록 등 다양한 정보를 참조하며 복잡한 판단이 요구된다.

- ① 전처리 과정에서 촬영 조건과 환자 정보를 정리하는 작업은 동일한 연산의 반복이 필요하므로, GPU의 병렬 처리 구조를 활용하는 것이 효과적이다.
- ② 수천 장의 이미지에 동일한 필터를 적용하여 윤곽을 추출하는 연산은 데이터마다 서로 다른 명령어를 수행해야 하므로, CPU의 고성능 코어를 활용하는 것이 적합하다.
- ③ 밝기 값의 기준치 초과 여부에 따라 보정 연산의 수행 여부가 달라지는 부분에서는 스레드마다 실행 경로가 동일하게 유지되므로, GPU의 병렬성이 온전히 보존된다.
- ④ 윤곽 추출 단계에서의 조건 분기는 스레드 블록 내 일부 스레드의 대기를 유발하여 병렬성 손실이 발생할 수 있으므로, 해당 시스템은 이 구간에서 GPU의 성능 저하를 감수해야 한다.
- ⑤ 환자별 보고서를 생성하는 단계는 단순 반복 연산의 비중이 높으므로, GPU에 전적으로 위임하는 것이 시스템 전체의 연산 효율을 극대화하는 방법이다.

[9~16] 다음 글을 읽고 물음에 답하시오.

최근까지 일반적인 컴퓨터 환경에서는 운영 체제를 제어하거나 사용자 인터페이스를 처리하고, 문서를 작성하거나 웹을 탐색하는 등 다양한 범용 작업이 주요 연산 대상이었다. 이러한 작업은 명령어의 종류가 많고 그 순서가 수시로 바뀌며, 연산 간 종속성이 크다는 특징이 있다. 컴퓨터의 연산은 주로 여러 개의 코어로 구성된 중앙 처리 장치(CPU)에서 수행된다. 코어는 컴퓨터의 두뇌 역할을 하는 것으로 명령어를 해석하고 각종 연산을 실행한다. CPU는 여러 개의 ㉠[고성능 코어]로 구성되어 복잡한 제어 흐름을 정밀하게 조율할 수 있도록 설계되었다. 그러나 인공 지능 학습, 고해상도 영상 처리, 과학 기술 관련 계산 등 대규모 데이터를 대상으로 동일한 연산을 반복 수행해야 하는 작업이 증가하면서 기존의 연산 환경에 변화가 생겼다. CPU는 코어 수에 한계가 있고 개별 코어가 복잡한 제어 흐름에 맞추어 다양한 연산을 처리하도록 설계되어 있기 때문에 동일한 연산을 대량으로 반복 수행해야 하는 현대의 병렬 연산 환경을 효율적으로 처리하기 어렵다. 이러한 상황에서 컴퓨터의 새로운 중심 장치로 떠오른 것이 그래픽 처리 장치(GPU)이다.

GPU는 컴퓨터 내에서 그래픽 처리를 담당하는 장치로, 수많은 픽셀에 동일한 연산이 반복적으로 이루어지는 그래픽 연산의 특성상 다수의 코어가 병렬로 작동하는 구조로 이루어져 있다. GPU는 수백 개에서 수천 개에 이르는 단순 연산 전용 코어를 하나의 칩에 집적한 상태로, 많은 일꾼이 한 가지 작업을 동시에 수행하는 공장과 같다. 이는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있으며, GPU의 코어는 제어 기능을 최소화하고, 동일한 연산을 빠르게 수행하도록 설계되어 있다. 이러한 구조를 바탕으로 GPU는 원래의 목적인 그래픽 처리 외에도 다양한 분야에서 활용되고 있다. 인공 지능 학습에서는 행렬 곱셈과 역전파 계산 등 동일한 연산의 반복이 이루어지며, 영상 처리나 고해상도 그래픽 구현에서는 수많은 픽셀 단위의 연산이 병렬로 적용된다. 또한 기상 예측, 유전자 배열 분석, 유체 역학 시뮬레이션 등에서도 대규모 수치 연산을 빠르게 처리해야 하기 때문에, ㉡[병렬 구조에 기반한 연산 장치]가 필요하다.

GPU 내부에는 수많은 연산 코어가 집적되어 있고 이들은 일정 단위로 묶여 연산 유닛을 형성한다. 연산 유닛은 여러 개의 스레드를 병렬로 실행할 수 있는 독립적인 실행 공간으로, 공장 내에서 같은 작업을 수행하는 작업반에 비유할 수 있다. 스레드는 연산을 수행하는 최소 단위로, 하나의 스레드는 하나의 명령어를 실행한다. GPU는 여러 스레드를 하나의 스레드 블록으로 묶어서 관리하는데, 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행하게 된다. 명령어가 GPU에 전달되면 디코딩 과정을 통해 해석되며, 스케줄러는 이 명령어에 따라 스레드 블록을 각 연산 유닛에 분배하고 병렬 처리가 이루어지도록 제어한다. ㉢[만약 연산량이 균등하게 분배되지 않으면 일부 연산 유닛은 다른 유닛의 연산이 끝날 때까지 대기 상태에 놓이게 되어 전체 GPU의 자원이 비효율적으로 사용되는 문제가 발생할 수 있다.] 분배가 끝나면 스레드 블록은 병렬로 연산을 실행하고, 실행한 결과는 GPU 내의 메모리에 저장된다.

GPU는 연산 속도를 높이기 위해 ㉣[계층화된 메모리 구조]를 갖추고 있다. 메모리는 접근 속도, 용량, 사용 범위 등에 따라 전역 메모리, 공유 메모리, 레지스터로 나뉜다. 전역 메모리는 모든 스레드가 접근할 수 있는 대용량 저장 공간으로, 다음 단계의 연산이나 외부와의 공유를 위해 연산 실행 결과를 저장한다. 다만

접근 속도가 느리다는 단점이 있기 때문에 연산 중에 사용하기에는 적합하지 않다. 따라서 연산 중에는 개별 스레드만 사용할 수 있는 저장소인 레지스터나 같은 스레드 블록 내의 스레드들이 공동으로 사용하는 공유 메모리를 활용하여 데이터를 저장하고 처리하는 방식으로 성능을 최적화한다. 레지스터는 용량은 작지만 접근이 매우 빠르고, 공유 메모리는 전역 메모리보다 빠르면서도 여러 스레드 간의 정보 공유가 가능하다는 장점이 있다. 다만, 같은 스레드 블록 내의 여러 스레드가 동시에 같은 메모리 공간에 접근할 경우, 충돌이 발생해 병렬성이 저하되어 연산이 지연되거나 처리 속도가 저하되는 현상이 나타날 수 있다. 따라서 메모리의 효율적 분배와 접근 방식의 조정은 GPU 연산 성능에 중요한 영향을 미친다.

한편, GPU는 동일한 명령어를 다수의 데이터에 일괄 적용하는 방식에 최적화되어 있지만, 분기문이나 조건 판단이 자주 등장하는 연산에서는 병렬 처리의 효율이 낮아질 수 있다. 조건 판단은 특정 수치나 상태에 따라 연산의 흐름을 다르게 결정하는 과정이며, 분기문은 이러한 조건 판단에 따라 실행 경로를 분리하거나 연결하는 명령 구조를 의미한다. 예컨대, '어떤 값이 일정 기준보다 크면 A 연산을, 그렇지 않으면 B 연산을 수행하라.'는 명령이 있을 경우, 스레드마다 실행해야 할 작업이 달라진다. GPU는 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있기 때문에, 이러한 분기문이 삽입되면 일부 스레드는 다른 스레드의 작업이 끝날 때까지 대기하게 되고, 그 결과 병렬성이 손실되는 현상이 발생한다. ㉤[따라서 GPU는 단순하고 반복적인 연산에는 매우 효과적이지만, 연산 경로가 자주 갈라지거나 복잡한 조건 판단이 요구되는 작업에서는 성능이 저하될 수 있다.]

CPU와 GPU의 장점을 고려하여, 최근의 컴퓨팅 시스템은 두 장치의 기능을 나누어 운용하는 구조로 발전하고 있다. 각 장치의 강점을 바탕으로 전체 시스템을 구성하여 연산 효율을 높이는 것이다. 연산 작업의 성격과 목적에 따라 CPU와 GPU 간 연산 흐름을 유기적으로 조정할 수 있도록 하드웨어 및 소프트웨어 설계 방식이 변화하고, 두 연산 장치의 성능을 최대한으로 끌어내면서도 효과적인 활용을 위한 프로그래밍이 이루어지고 있다. 이렇게 현대의 컴퓨팅 시스템은 두 연산 장치가 각자의 구조적 강점을 최대한 발휘할 수 있도록 하여 더욱 복잡하고 대규모화된 연산 환경에 대응하고 있다.

* 픽셀: 주조화될 수 있는 화면의 가장 작은 단위. 작은 점의 행과 열로 이루어져 있는 화면의 작은 점 각각을 이르는 말.

9. 밑줄의 문단별 기능에 대한 설명으로 적절하지 않은 것은?

- ㉠ 1문단은 기존 연산 환경의 특징을 제시하고, 새로운 연산 수요의 등장에 따라 기존 장치의 한계가 드러나는 상황을 서술하고 있다.
- ㉡ 2문단은 GPU의 내부 구조적 특성을 설명하고, 이를 바탕으로 그래픽 처리 이외의 분야에서도 GPU가 활용되는 사례를 나열하고 있다.
- ㉢ 3문단은 GPU 내부에서 연산이 분배되고 실행되는 과정을 순차적으로 서술하면서, 분배 과정에서 발생할 수 있는 문제를 언급하고 있다.
- ㉣ 4문단은 GPU의 메모리 구조를 접근 속도와 사용 범위에 따라 유형화하여 설명하면서, 각 메모리 유형이 상호 대체 가능함을 밝히고 있다.
- ㉤ 5문단은 GPU의 구조적 특성에서 비롯되는 한계를 구체적인 예시와 함께 서술하고, 이로 인해 특정 유형의 연산에서 성능이 저하될 수 있음을 설명하고 있다.

10. 밑줄에 대한 이해로 적절한 것만을 <보기>에서 있는 대로 고른 것은?

<보기>

- ㄱ. 스레드는 연산을 수행하는 최소 단위이며, 복수의 스레드가 하나의 스레드 블록으로 묶여 관리되는데, 같은 블록 내의 스레드들은 서로 다른 명령어를 각각 독립적으로 실행한다.
- ㄴ. GPU가 그래픽 처리 이외의 분야에서도 활용될 수 있는 것은, 해당 분야의 연산이 동일한 연산을 다수의 데이터에 반복 적용하는 특성을 공유하기 때문이다.
- ㄷ. 전역 메모리는 모든 스레드가 접근할 수 있지만 접근 속도가 느리고, 레지스터는 접근이 매우 빠르지만 개별 스레드만 사용할 수 있으므로, 두 메모리는 접근 속도와 사용 범위 사이에서 상반된 특성을 보인다.

- ㉠ ㄱ
- ㉡ ㄴ
- ㉢ ㄱ, ㄷ
- ㉣ ㄴ, ㄷ
- ㉤ ㄱ, ㄴ, ㄷ

11. ㉠과 ㉡에 대한 이해로 가장 적절한 것은?

- ㉠ ㉠은 동일한 연산의 대량 반복에 최적화된 코어이며, ㉡은 ㉠의 구조적 특성을 계승하여 개별 코어의 제어 기능을 강화한 장치이다.
- ㉡ ㉡은 복잡한 제어 흐름을 정밀하게 조율하기 위한 코어이며, ㉠의 필요성은 ㉡만으로는 충족할 수 없는 새로운 연산 수요가 등장한 데서 비롯된다.
- ㉢ ㉢은 다수의 데이터에 서로 다른 명령어를 개별 적용하는 방식에 적합한 코어이며, ㉠은 ㉢의 이러한 방식을 대규모로 확장한 장치이다.
- ㉣ ㉢의 수를 늘려 GPU 수준의 코어 집적도를 달성하면 ㉡의 필요성이 사라지므로, ㉠은 ㉢의 수적 한계를 보완하기 위한 임시적 대안이다.
- ㉤ ㉠과 ㉡의 내부 코어는 동일한 구조와 성능을 가지며, 차이는 오직 하나의 칩에 집적된 코어의 수에서만 발생한다.

12. ㉢에 대한 추론으로 가장 적절한 것은?

- ㉠ ㉢의 문제는 스레드 블록 내부에서 스레드 간의 명령어가 상이하기 때문에 발생하며, 분기문의 삽입과 동일한 원인에서 비롯된다.
- ㉡ ㉢의 문제가 발생하면 대기 상태에 놓인 연산 유닛의 스레드 블록은 해당 유닛에서 제거되고 다른 유닛으로 재분배된다.
- ㉢ ㉢의 문제는 스케줄러가 스레드 블록을 연산 유닛에 분배하는 단계에서 발생하는 것이므로, 분배 이후의 병렬 연산 실행 단계에서는 동일한 문제가 나타나지 않는다.
- ㉣ ㉢에서 대기 상태에 놓인 연산 유닛은 자신에게 할당된 연산을 이미 완료한 상태이며, 다른 유닛의 연산이 끝나기를 기다리는 동안 추가적인 자원을 소모하지 않는다.
- ㉤ ㉢의 문제와 5문단에서 서술된 분기문에 의한 병렬성 손실은, 일부 실행 단위가 다른 실행 단위의 작업 완료를 기다리며 유휴 상태에 놓인다는 점에서 구조적으로 유사하나, ㉢은 연산 유닛 간의 연산량 불균등에서 비롯되고 분기문에 의한 손실은 스레드 블록 내 스레드 간의 실행 경로 분리에서 비롯된다는 점에서 발생 층위가 다르다.

13. ㉢에 대한 이해로 적절하지 않은 것은?

- ㉠ ㉢은 접근 속도, 용량, 사용 범위가 서로 다른 메모리들을 단계적으로 배치한 구조이다.
- ㉡ ㉢을 구성하는 메모리 중 전역 메모리는 용량이 크고 모든 스레드가 접근 가능하지만, 연산 중에 사용하기에는 속도 면에서 불리하다.
- ㉢ ㉢에서 레지스터와 공유 메모리가 연산 중에 활용되는 것은, 전역 메모리의 느린 접근 속도를 보완하기 위한 것이다.
- ㉣ ㉢에서 공유 메모리는 같은 스레드 블록 내의 스레드들이 공동으로 사용하므로, 서로 다른 스레드 블록에 속한 스레드들 간에도 공유 메모리를 통해 데이터를 교환할 수 있다.
- ㉤ ㉢의 효율적 운용을 위해서는 각 메모리 유형의 특성에 맞게 데이터를 분배하고, 접근 방식을 적절히 조정하는 것이 필요하다.

14. ㉔가 글의 논지 전개에서 수행하는 역할로 가장 적절한 것은?

- ① ㉔는 5문단에서 전개된 논의를 종합하여 GPU의 구조적 강점과 한계를 동시에 제시하면서, GPU가 자체적으로 한계를 극복할 수 있으므로 6문단에서 서술되는 CPU와의 협력적 운용이 불필요하다는 논거를 뒷받침하는 기능을 한다.
- ② ㉔는 5문단에서 전개된 논의를 종합하여 GPU의 구조적 강점과 한계를 동시에 제시하면서, 6문단에서 CPU와 GPU의 상호 보완적 운용이라는 결론으로 나아가기 위한 논리적 교두보로 기능한다.
- ③ ㉔는 5문단에서 분기문이 초래하는 병렬성 손실을 서술한 흐름을 마무리하면서, 이후 서술에서 분기문 최적화 기술의 발전 방향을 제시하기 위한 문제 제기로 기능한다.
- ④ ㉔는 5문단에서 분기문이 초래하는 병렬성 손실을 서술한 흐름을 마무리하면서, GPU의 한계가 설계 구조상 근본적으로 극복 불가능함을 단정하여 새로운 연산 장치의 필요성을 시사하는 기능을 한다.
- ⑤ ㉔는 4문단에서 제시한 메모리 충돌 문제의 해결 방안을 종합적으로 제시하면서, 5문단 전체의 논의를 메모리 관리라는 주제로 수렴시키는 기능을 한다.

15. 뒷글을 바탕으로 <보기>를 이해한 내용으로 적절하지 않은 것은?

<보기>

어느 물류 센터에서는 하루에 수만 건의 택배 상자를 분류하는 자동화 시스템을 운용한다. 이 시스템은 크게 세 단계로 이루어져 있다. 첫째, 컨베이어 벨트 위의 모든 상자에 부착된 바코드를 스캔하여, 각 상자의 무게와 크기를 동일한 공식에 대입해 배송 요금을 산출하는 단계이다. 이 단계에서는 수만 건의 상자에 대해 같은 계산이 반복된다. 둘째, 요금 산출이 완료된 상자 중 일부에서 바코드 인식 오류가 발생한 경우, 해당 상자만을 별도의 라인으로 분리하여 수동 검수를 진행하는 단계이다. 이때 오류가 발생한 상자과 정상 상자는 서로 다른 처리 경로를 거치게 된다. 셋째, 모든 분류가 완료된 후, 각 배송 권역별로 상자를 재배치하면서 권역별 차량 적재 순서와 배송 경로를 최적화하는 단계이다. 이 단계에서는 각 권역의 교통 상황, 배송 우선순위, 차량 용량 등 다양한 변수를 종합적으로 고려해야 한다.

- ① 첫째 단계에서 수만 건의 상자에 동일한 공식을 반복 적용하여 배송 요금을 산출하는 연산은 GPU의 병렬 처리 구조에 적합한 작업에 해당한다.
- ② 둘째 단계에서 바코드 인식 오류 여부에 따라 상자의 처리 경로가 달라지는 상황은, GPU의 스레드 블록 내에서 분기문이 삽입되었을 때 일부 스레드가 대기하게 되는 상황과 유사한 구조를 지닌다.
- ③ 셋째 단계에서 교통 상황, 배송 우선순위, 차량 용량 등 다양한 변수를 종합적으로 고려하는 작업은 명령어의 종류가 많고 연산 간 종속성이 큰 작업의 성격을 지니므로, CPU의 구조적 강점이 발휘될 수 있는 영역이다.
- ④ 첫째 단계의 연산을 GPU가, 셋째 단계의 연산을 CPU가 각각 담당하도록 설계하는 것은, 연산 작업의 성격과 목적에 따라 두 장치 간 연산 흐름을 유기적으로 조정하는 방식에 해당한다.
- ⑤ 둘째 단계에서 오류가 발생한 상자를 별도의 라인으로 분리하는 것은, GPU의 전역 메모리에서 공유 메모리로 데이터를 이동시켜 접근 속도를 높이는 것과 동일한 원리에 기반한 것이다.

16. 뒷글을 바탕으로 <보기>의 ㉔와 ㉕에 대해 추론한 내용으로 가장 적절한 것은?

<보기>

어느 연구팀이 동일한 대규모 행렬 곱셈 연산을 두 가지 시스템에서 각각 수행하는 실험을 진행하였다. ㉔[시스템 X]는 소수의 고성능 코어로 구성된 장치를 중심으로 설계되어, 복잡한 제어 흐름을 정밀하게 조율하면서 연산을 순차적으로 처리하는 방식을 취한다. ㉕[시스템 Y]는 수천 개의 단순 연산 전용 코어를 하나의 칩에 집적한 장치를 중심으로 설계되어, 스케줄러가 연산을 다수의 연산 유닛에 분배하고 스레드 블록 단위로 병렬 처리하는 방식을 취한다. 실험 결과, 단순 반복 연산 구간에서는 ㉕의 처리 속도가 ㉔보다 현저히 빨랐으나, 연산 중간에 조건 분기가 빈번하게 삽입된 구간에서는 ㉕의 처리 속도가 크게 저하되어 ㉔와의 차이가 줄어들었다.

- ① 단순 반복 연산 구간에서 ㉕가 ㉔보다 빨랐던 것은, ㉕의 개별 코어가 ㉔의 개별 코어보다 높은 연산 성능을 갖추고 있기 때문이다.
- ② 조건 분기가 빈번한 구간에서 ㉕의 처리 속도가 저하된 것은, ㉕의 전역 메모리 접근 속도가 조건 분기 삽입 시에 추가적으로 느려지기 때문이다.
- ③ 조건 분기가 빈번한 구간에서 ㉔와 ㉕의 처리 속도 차이가 줄어든 것은, 해당 구간에서 ㉕의 스레드 블록 내 일부 스레드가 대기 상태에 놓여 병렬성이 손실되는 반면 ㉔는 복잡한 제어 흐름을 처리할 수 있는 구조를 갖추고 있기 때문이다.
- ④ ㉔의 장치가 대규모 행렬 곱셈 연산 전체에서 ㉕보다 낮은 처리 속도를 보인 것은, ㉔의 장치가 메모리 계층 구조를 갖추고 있지 않기 때문이다.
- ⑤ 단순 반복 연산 구간에서 ㉕가 빠른 처리 속도를 보인 것은, ㉕의 스케줄러가 조건 분기를 자동으로 제거하여 모든 스레드가 동일한 경로로 연산을 수행하도록 조정하였기 때문이다.

[17~25] 다음 글을 읽고 물음에 답하시오.

최근까지 일반적인 컴퓨터 환경에서는 운영 체제를 제어하거나 사용자 인터페이스를 처리하고, 문서를 작성하거나 웹을 탐색하는 등 다양한 범용 작업이 주요 연산 대상이었다. 이러한 작업은 명령어의 종류가 많고 그 순서가 수시로 바뀌며, 연산 간 종속성이 크다는 특징이 있다. 컴퓨터의 연산은 주로 여러 개의 코어로 구성된 중앙 처리 장치(CPU)에서 수행된다. 코어는 컴퓨터의 두뇌 역할을 하는 것으로 명령어를 해석하고 각종 연산을 실행한다. CPU는 여러 개의 고성능 코어로 구성되어 복잡한 제어 흐름을 정밀하게 조율할 수 있도록 설계되었다. 그러나 인공 지능 학습, 고해상도 영상 처리, 과학 기술 관련 계산 등 대규모 데이터를 대상으로 동일한 연산을 반복 수행해야 하는 작업이 증가하면서 기존의 연산 환경에 변화가 생겼다. CPU는 코어 수에 한계가 있고 개별 코어가 복잡한 제어 흐름에 맞추어 다양한 연산을 처리하도록 설계되어 있기 때문에 동일한 연산을 대량으로 반복 수행해야 하는 현대의 병렬 연산 환경을 효율적으로 처리하기 어렵다. 이러한 상황에서 컴퓨터의 새로운 중심 장치로 떠오른 것이 그래픽 처리 장치(GPU)이다.

GPU는 컴퓨터 내에서 그래픽 처리를 담당하는 장치로, 수많은 픽셀에 동일한 연산이 반복적으로 이루어지는 그래픽 연산의 특성상 다수의 코어가 병렬로 작동하는 구조로 이루어져 있다. GPU는 수백 개에서 수천 개에 이르는 단순 연산 전용 코어를 하나의 칩에 집적한 상태로, 많은 일꾼이 한 가지 작업을 동시에 수행하는 공장과 같다. ㉠[이는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있으며, GPU의 코어는 제어 기능을 최소화하고, 동일한 연산을 빠르게 수행하도록 설계되어 있다.] 이러한 구조를 바탕으로 GPU는 원래의 목적인 그래픽 처리 외에도 다양한 분야에서 활용되고 있다. 인공 지능 학습에서는 행렬 곱셈과 역전파 계산 등 동일한 연산의 반복이 이루어지며, 영상 처리나 고해상도 그래픽 구현에서는 수많은 픽셀 단위의 연산이 병렬로 적용된다. 또한 기상 예측, 유전자 배열 분석, 유체 역학 시뮬레이션 등에서도 대규모 수치 연산을 빠르게 처리해야 하기 때문에, 병렬 구조에 기반한 연산 장치가 필요하다.

GPU 내부에는 수많은 연산 코어가 집적되어 있고 이들은 일정 단위로 묶여 연산 유닛을 형성한다. 연산 유닛은 여러 개의 스레드를 병렬로 실행할 수 있는 독립적인 실행 공간으로, 공장 내에서 같은 작업을 수행하는 작업반에 비유할 수 있다. 스레드는 연산을 수행하는 최소 단위로, 하나의 스레드는 하나의 명령어를 실행한다. GPU는 여러 스레드를 하나의 스레드 블록으로 묶어서 관리하는데, 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행하게 된다. 명령어가 GPU에 전달되면 디코딩 과정을 통해 해석되며, ㉡[스케줄러]는 이 명령어에 따라 스레드 블록을 각 연산 유닛에 분배하고 병렬 처리가 이루어지도록 제어한다. 만약 연산량이 균등하게 분배되지 않으면 일부 연산 유닛은 다른 유닛의 연산이 끝날 때까지 대기 상태에 놓이게 되어 전체 GPU의 자원이 비효율적으로 사용되는 문제가 발생할 수 있다. 분배가 끝나면 스레드 블록은 병렬로 연산을 실행하고, 실행한 결과는 GPU 내의 메모리에 저장된다.

GPU는 연산 속도를 높이기 위해 계층화된 메모리 구조를 갖추고 있다. 메모리는 접근 속도, 용량, 사용 범위 등에 따라 ㉢[전역 메모리], 공유 메모리, 레지스터로 나뉜다. 전역 메모리는 모든 스레드가 접근할 수 있는 대용량 저장 공간으로, 다음 단계의 연산이나 외부와의 공유를 위해 연산 실행 결과를 저장한다. 다만 접근 속도가 느리다는 단점이 있기 때문에 연산 중에 사용하기에

는 적합하지 않다. 따라서 연산 중에는 개별 스레드만 사용할 수 있는 저장소인 레지스터나 같은 스레드 블록 내의 스레드들이 공동으로 사용하는 공유 메모리를 활용하여 데이터를 저장하고 처리하는 방식으로 성능을 최적화한다. 레지스터는 용량은 작지만 접근이 매우 빠르고, 공유 메모리는 전역 메모리보다 빠르면서도 여러 스레드 간의 정보 공유가 가능하다는 장점이 있다. 다만, 같은 스레드 블록 내의 여러 스레드가 동시에 같은 메모리 공간에 접근할 경우, 충돌이 발생해 병렬성이 저하되어 연산이 지연되거나 처리 속도가 저하되는 현상이 나타날 수 있다. ㉣[따라서 메모리의 효율적 분배와 접근 방식의 조정은 GPU 연산 성능에 중요한 영향을 미친다.]

한편, GPU는 동일한 명령어를 다수의 데이터에 일괄 적용하는 방식에 최적화되어 있지만, 분기문이나 조건 판단이 자주 등장하는 연산에서는 병렬 처리의 효율이 낮아질 수 있다. ㉤[조건 판단]은 특정 수치나 상태에 따라 연산의 흐름을 다르게 결정하는 과정이며, 분기문은 이러한 조건 판단에 따라 실행 경로를 분리하거나 연결하는 명령 구조를 의미한다. 예컨대, '어떤 값이 일정 기준보다 크면 A 연산을, 그렇지 않으면 B 연산을 수행하라.'는 명령이 있을 경우, 스레드마다 실행해야 할 작업이 달라진다. GPU는 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있기 때문에, 이러한 분기문이 삽입되면 일부 스레드는 다른 스레드의 작업이 끝날 때까지 대기하게 되고, 그 결과 병렬성이 손실되는 현상이 발생한다. 따라서 GPU는 단순하고 반복적인 연산에는 매우 효과적이지만, 연산 경로가 자주 갈라지거나 복잡한 조건 판단이 요구되는 작업에서는 성능이 저하될 수 있다.

CPU와 GPU의 장점을 고려하여, 최근의 컴퓨팅 시스템은 두 장치의 기능을 나누어 운용하는 구조로 발전하고 있다. 각 장치의 강점을 바탕으로 전체 시스템을 구성하여 연산 효율을 높이는 것이다. 연산 작업의 성격과 목적에 따라 CPU와 GPU 간 연산 흐름을 유기적으로 조정할 수 있도록 하드웨어 및 소프트웨어 설계 방식이 변화하고, 두 연산 장치의 성능을 최대한으로 끌어내면서도 효과적인 활용을 위한 프로그래밍이 이루어지고 있다. ㉥[이렇게 현대의 컴퓨팅 시스템은 두 연산 장치가 각자의 구조적 강점을 최대한 발휘할 수 있도록 하여 더욱 복잡하고 대규모화된 연산 환경에 대응하고 있다.]

* 픽셀: 주조화될 수 있는 화면의 가장 작은 단위. 작은 점의 행과 열로 이루어져 있는 화면의 작은 점 각각을 이르는 말.

17. 밑글에서 다루고 있는 중심 화제로 가장 적절한 것은?

- ① 그래픽 처리 장치의 메모리 구조 개선을 위한 기술적 방안
- ② 중앙 처리 장치의 코어 수 확장이 연산 효율에 미치는 영향
- ③ 연산 환경 변화에 따른 그래픽 처리 장치의 부상과 그 특성 및 한계
- ④ 병렬 연산 방식과 순차 연산 방식의 처리 속도에 대한 정량적 비교
- ⑤ 스레드 블록의 분배 알고리즘 최적화를 통한 연산 성능 향상 전략

18. 밑줄에 대한 이해로 적절한 것은?

- ① CPU의 코어와 GPU의 코어는 모두 명령어를 해석하고 연산을 실행하는 기능을 수행하지만, CPU의 코어는 코어당 처리할 수 있는 데이터의 양이 GPU의 코어보다 적다.
- ② 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행하며, 이는 GPU가 서로 다른 연산을 동시에 처리하는 방식에 최적화되어 있기 때문이다.
- ③ GPU의 연산 유닛은 여러 개의 스레드를 병렬로 실행할 수 있는 독립적인 실행 공간으로, 명령어의 디코딩은 각 연산 유닛 내부에서 개별적으로 이루어진다.
- ④ GPU가 그래픽 처리 외의 분야에서도 활용될 수 있는 것은, 해당 분야들이 대규모 데이터에 대해 동일한 연산을 반복 수행해야 한다는 공통점을 지니기 때문이다.
- ⑤ 현대의 컴퓨팅 시스템에서 CPU는 단순 반복 연산을, GPU는 복잡한 제어 흐름이 요구되는 연산을 각각 전담하도록 역할이 고정되어 있다.

19. ㉔를 바탕으로 추론한 내용으로 적절하지 않은 것은?

- ① GPU가 인공지능 학습에서 활용될 수 있는 것은, 행렬 곱셈과 역전파 계산이 ㉔에서 서술한 GPU의 최적화 방식에 부합하는 연산이기 때문이다.
- ② ㉔에서 GPU의 코어가 제어 기능을 최소화했다는 것은, 복잡한 명령어 해석이나 분기 처리보다는 단일 연산의 신속한 반복 실행에 자원을 집중했음을 의미한다.
- ③ ㉔에서 서술한 GPU의 설계 원리는, 5문단에서 분기문이 삽입될 때 병렬성이 손실되는 현상이 발생하는 구조적 원인으로 작용한다.
- ④ ㉔에서 동일한 명령어를 다수의 데이터에 반복 적용한다는 것은, 각 코어가 서로 다른 명령어를 독립적으로 해석하여 개별 데이터에 맞춤형 연산을 수행함을 의미한다.
- ⑤ ㉔에서 서술한 GPU의 설계 특성은 CPU의 고성능 코어가 복잡한 제어 흐름을 정밀하게 조율하는 설계 특성과 대비되는 것으로, 두 장치가 서로 다른 유형의 연산에 적합하게 설계되었음을 보여준다.

20. ㉑의 기능에 대한 이해로 가장 적절한 것은?

- ① ㉑은 각 연산 유닛 내부에서 스레드 간의 명령어 실행 순서를 결정하여, 동일 블록 내 스레드들이 서로 다른 명령어를 번갈아 실행하도록 조율하는 역할을 한다.
- ② ㉑은 GPU에 전달된 명령어를 직접 해석하는 디코딩 기능을 수행하며, 해석된 명령어를 스레드 블록으로 변환하여 연산 유닛에 전달하는 역할을 한다.
- ③ ㉑은 디코딩 과정을 거쳐 해석된 명령어에 따라 스레드 블록을 각 연산 유닛에 분배하여 병렬 처리가 이루어지도록 제어하는 역할을 한다.
- ④ ㉑은 연산이 완료된 후 각 연산 유닛의 결과를 수집하여 전역 메모리에 기록하는 역할을 수행하며, 메모리 계층 간 데이터 이동을 관리하는 역할을 한다.
- ⑤ ㉑은 분기문이 삽입된 연산에서 스레드 블록 내의 스레드들을 실행 경로별로 재분류하여, 동일한 경로를 선택한 스레드들만을 묶어 별도의 블록으로 재구성하는 역할을 한다.

21. ㉒에 대해 추론한 내용으로 가장 적절한 것은?

- ① ㉒은 접근 속도가 빠르지만 용량이 작아, 연산 중에 스레드들이 즉각적으로 데이터를 읽고 쓰는 용도로 사용된다.
- ② ㉒은 같은 스레드 블록 내의 스레드들만 접근할 수 있으므로, 서로 다른 연산 유닛에 분배된 스레드 블록들은 ㉒을 통해 데이터를 교환할 수 없다.
- ③ ㉒은 연산 실행 결과를 저장하여 다음 단계의 연산이나 외부와의 공유에 활용되지만, 연산 중에 빈번하게 접근하면 병렬성이 저하될 수 있으므로 연산 중에는 레지스터나 공유 메모리가 대신 활용된다.
- ④ ㉒의 접근 속도가 느린 문제는 공유 메모리를 경유하여 데이터를 전달하는 방식으로 완전히 해소되므로, GPU의 연산 성능은 ㉒의 속도와 무관하게 결정된다.
- ⑤ ㉒은 개별 스레드만 사용할 수 있는 전용 저장소이므로, 한 스레드가 ㉒에 저장한 데이터를 다른 스레드가 읽으려면 반드시 공유 메모리를 거쳐야 한다.

22. ㉔가 글의 논증에서 수행하는 기능으로 가장 적절한 것은?

- ① ㉔는 4문단에서 전개된 메모리 유형별 특성과 충돌 가능성에 대한 논의를 종합하여, GPU의 현행 메모리 계층 구조 자체를 전면 개편해야 한다는 결론을 도출하는 기능을 한다.
- ② ㉔는 4문단에서 전개된 메모리 유형별 특성과 충돌 가능성에 대한 논의를 종합하여, 메모리의 물리적 구조 자체가 아니라 그 운용 방식이 GPU 성능의 관건임을 귀결로 제시하는 기능을 한다.
- ③ ㉔는 공유 메모리에서 발생할 수 있는 충돌 문제가 서술된 뒤에 위치하여, 전역 메모리의 접근 속도를 향상시키는 것이 충돌 문제의 유일한 해결 방안임을 제시하는 기능을 한다.
- ④ ㉔는 공유 메모리에서 발생할 수 있는 충돌 문제가 서술된 뒤에 위치하여, 레지스터와 공유 메모리가 기능적으로 동일하므로 두 유형을 통합해야 한다는 결론을 도출하는 기능을 한다.
- ⑤ ㉔는 3문단에서 다른 스레드 블록의 불균등 분배 문제와 4문단의 메모리 충돌 문제가 동일한 원인에서 비롯됨을 밝히면서, 두 문제를 통합적으로 해결하는 방안을 제안하는 기능을 한다.

23. ㉔를 이해한 내용으로 가장 적절한 것은?

- ① ㉔는 CPU의 구조적 한계가 극복되어 GPU 없이도 모든 연산 환경에 대응할 수 있게 되었음을 서술한 것이다.
- ② ㉔는 GPU가 CPU의 기능을 완전히 흡수하여 단일 장치로 모든 연산을 처리하는 시스템이 구현되었음을 서술한 것이다.
- ③ ㉔에서 '각자의 구조적 강점'이란, CPU의 경우 복잡한 제어 흐름의 조율에, GPU의 경우 동일한 연산의 대량 병렬 처리에 해당한다.
- ④ ㉔에서 '더욱 복잡하고 대규모화된 연산 환경'이란, 단순 반복 연산만이 대폭 증가한 상황을 가리키며, 이에 대응하기 위해 GPU의 코어 수를 늘리는 것이 핵심 전략이다.
- ⑤ ㉔는 CPU와 GPU를 동시에 운용할 경우 두 장치 간의 연산 흐름 충돌이 불가피하다는 문제를 지적하면서, 이를 해결하기 위한 향후 과제를 제시한 것이다.

24. 뒷글을 바탕으로 <보기>를 이해한 내용으로 적절하지 않은 것은?

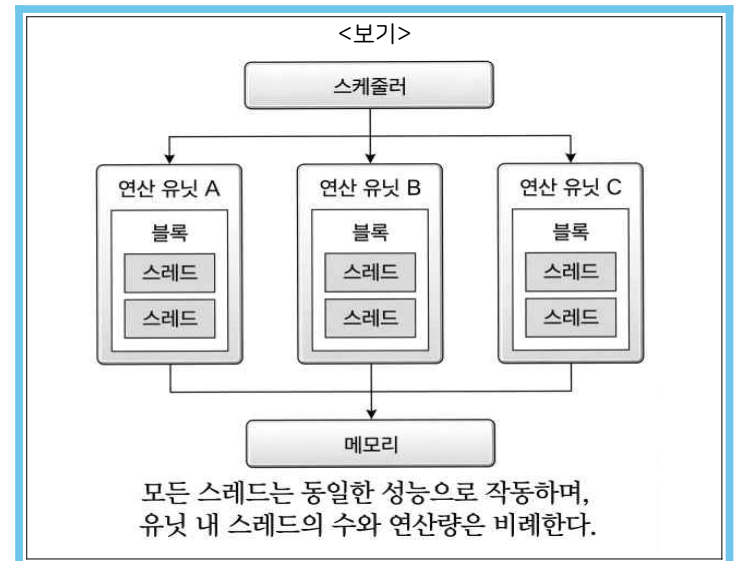
<보기>

한 자율 주행 시스템은 차량에 장착된 다수의 카메라로부터 실시간으로 영상 데이터를 수신하여, 주변 환경을 인식하고 주행 판단을 내리는 작업을 수행한다. 이 시스템의 작업은 크게 네 가지 단계로 나뉜다. 1단계에서는 여러 카메라에서 동시에 들어오는 영상 프레임 각각에 대해 동일한 필터를 적용하여 노이즈를 제거하는 전처리를 수행한다. 2단계에서는 전처리된 영상에서 차선, 보행자, 신호등 등의 객체를 탐지하는데, 이 과정에서 수백만 개의 픽셀 데이터에 대해 동일한 신경망 연산이 반복적으로 적용된다. 3단계에서는 탐지된 객체들의 이동 방향과 속도를 예측하는데, 돌발적인 보행자 출현이나 신호 변경 등 예외적 상황이 감지되면 긴급 회피 연산으로 전환하고, 그렇지 않으면 일반 주행 연산을 계속하는 조건 분기가 포함된다. 4단계에서는 1~3단계의 결과를 종합하여, 현재 교통 상황, 목적지까지의 경로, 법규 준수 여부 등 다양한 변수를 고려한 최종 주행 판단을 내린다.

- ① 1단계에서 여러 카메라의 영상 프레임에 동일한 필터를 적용하는 작업은, GPU가 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 부합하므로 GPU의 병렬 처리 구조를 활용하기에 적합하다.

- ② 2단계에서 수백만 개의 픽셀 데이터에 동일한 신경망 연산을 반복 적용하는 과정은, GPU 내부의 다수의 연산 유닛에 스레드 블록이 분배되어 병렬로 연산을 실행하는 방식으로 처리될 수 있다.
- ③ 3단계에서 돌발 상황 감지 여부에 따라 긴급 회피 연산과 일반 주행 연산 중 하나를 선택하는 구조는, 스레드 블록 내 스레드들의 실행 경로를 갈라지게 하여 일부 스레드의 대기를 유발할 수 있는 조건에 해당한다.
- ④ 3단계의 조건 분기로 인해 GPU의 병렬성이 손실될 수 있으므로, 해당 구간의 연산을 CPU에 위임하는 것은 두 장치 간 연산 흐름을 유기적으로 조정하는 방식에 해당한다.
- ⑤ 4단계에서 다양한 변수를 종합하여 최종 주행 판단을 내리는 작업은, 동일한 연산의 반복이 핵심이므로 GPU의 스레드 블록 단위 병렬 처리에 전적으로 위임하는 것이 가장 효율적이다.

25. <보기>는 GPU의 내부 구조를 나타낸 것이다. 뒷글을 바탕으로, <보기>의 GPU에서 ㉔이 빈번하게 요구되는 연산이 수행될 경우에 대해 추론한 내용으로 가장 적절한 것은?



- ① 유닛 A의 블록 내 두 스레드가 ㉔에 의해 서로 다른 실행 경로를 따르게 될 경우, 스케줄러가 해당 블록을 해체하고 동일 경로의 스레드끼리 재 묶어 유닛 B, C에 재분배함으로써 병렬성 손실을 방지할 수 있다.
- ② ㉔이 빈번하게 요구되는 연산에서는, <보기>에서 모든 스레드가 동일한 성능으로 작동한다는 조건에 의해 각 스레드가 자신에게 할당된 경로의 연산을 독립적으로 완수할 수 있으므로, 유닛 A의 블록 내 병렬성은 온전히 보존된다.
- ③ ㉔이 빈번하게 요구되면 각 유닛 내 블록의 스레드들 사이에서 실행 경로가 갈라져 일부 스레드가 대기 상태에 놓이게 되며, 이로 인해 <보기>에서 유닛 내 스레드의 수와 연산량이 비례한다는 조건이 성립하더라도 각 유닛의 실질 처리 효율은 저하될 수 있다.
- ④ ㉔에 의해 스레드마다 실행 경로가 달라지면, 각 유닛에서 산출된 중간 결과가 하단의 메모리에 저장되는 과정에서 충돌이 발생하는 것이 ㉔으로 인한 성능 저하의 주된 원인이다.
- ⑤ ㉔이 빈번하게 요구되는 연산에서 병렬성이 손실되는 것은 유닛 A, B, C 간의 연산량 분배가 불균등해지기 때문이며, 이는 스케줄러의 분배 기능을 개선하면 해소될 수 있는 문제이다.

정답 및 해설

1. 정답: ②

정답 해설: 이 글은 1문단에서 대규모 데이터 대상 동일 연산 반복 작업의 증가라는 연산 환경 변화를 배경으로 제시하고, 2~4문단에서 GPU의 구조(코어·연산 유닛·스레드 블록)와 작동 원리(스케줄러 분배, 메모리 계층) 및 활용 분야를 설명한 뒤, 5문단에서 분기문에 따른 병렬성 손실이라는 한계를, 6문단에서 CPU와 GPU를 함께 운용하는 보완 방향을 서술하고 있다.

오답 해설:

- ① 두 장치의 내부 회로 구성을 단계적으로 제시하거나 전력 소비량을 비교하는 내용은 없다.
- ③ 동일한 실험 조건에서 처리 속도를 수치로 측정하여 비교하는 내용은 나타나지 않는다.
- ④ 여러 세대의 GPU를 시간순으로 나열하며 코어 수 증가 추이를 제시하는 내용은 없다.
- ⑤ 이론적 예측과 실제 성능 사이의 괴리를 분석하는 내용은 나타나지 않는다.

2. 정답: ④

정답 해설: 4문단에서 레지스터는 용량은 작지만 접근이 매우 빠르고, 공유 메모리는 전역 메모리보다 빠르면서도 여러 스레드 간 정보 공유가 가능하다고 하였다. 즉 레지스터가 접근 속도 면에서 가장 빠르고, 공유 메모리는 전역 메모리보다 빠르지만 레지스터보다 빠르다는 서술은 없다. 따라서 공유 메모리가 레지스터보다 접근 속도가 빠르다는 ④는 적절하지 않다.

오답 해설:

- ① 2문단에서 GPU의 코어는 제어 기능을 최소화하고 동일한 연산을 빠르게 수행하도록 설계되어 있다고 하였으므로 적절하다.
- ② 1문단에서 CPU가 사용되는 작업은 명령어의 종류가 많고 순서가 수시로 바뀌며 연산 간 종속성이 크다고 하였으므로 적절하다.
- ③ 3문단에서 분배가 끝나면 스레드 블록은 병렬로 연산을 실행하고 결과는 GPU 내의 메모리에 저장된다고 하였으므로 적절하다.
- ⑤ 6문단에서 연산 작업의 성격과 목적에 따라 하드웨어 및 소프트웨어 설계 방식이 변화하고 있다고 하였으므로 적절하다.

3. 정답: ⑤

정답 해설: <보기>에서 하단의 메모리는 모든 유닛과 연결된 전역 메모리에 해당한다. 4문단([B])에서 전역 메모리는 모든 스레드가 접근할 수 있는 대용량 저장 공간이지만 접근 속도가 느리기 때문에 연산 중에 사용하기에는 적합하지 않다고 하였다. 연산 중에는 접근이 빠른 레지스터나 공유 메모리를 활용하여 성능을 최적화한다고 하였으므로, 중간 결과를 하단의 메모리(전역 메모리)에 저장하여 처리하는 것이 성능 최적화에 유리하다는 ⑤는 적절하지 않다.

오답 해설:

- ① 3문단([A])에서 스케줄러가 스레드 블록을 각 연산 유닛에 분배한다고 하였고, <보기>에서 유닛 A~C에 블록이 하나씩 균등하게 배치되어 있으므로 적절하다.
- ② 3문단([A])에서 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행한다고 하였으므로 적절하다.
- ③ 4문단([B])에서 전역 메모리는 연산 실행 결과를 저장하며 다음 단계의 연산이나 외부와의 공유를 위해 사용된다고 하였고, <보기>에서 하단의 메모리는 모든 유닛과 연결되어 있으므로 적절하다.
- ④ 4문단([B])에서 같은 스레드 블록 내의 여러 스레드가 동시에 같은 메모리 공간에 접근하면 충돌이 발생하여 병렬성이 저하될 수 있다고 하였으므로 적절하다.

4. 정답: ⑤

정답 해설: ②는 CPU가 현대의 병렬 연산 환경을 효율적으로 처리하기 어렵다는 내용이다. 1문단에서 CPU의 개별 코어는 복잡한 제어 흐름을 정밀하게 조율하도록 설계되어 있으며 코어 수에 한계가 있다고 하였다. 한편 2문단에서 GPU의 코어는 제어 기능을 최소화하고 동일한 연산을 빠르게 수행하도록 설계되어 있으며, 수백에서 수천 개의 코어가 집적되어 있다고 하였는데, 이는 현대의 병렬 연산 환경이 요구하는 코어의 특성과 규모를 보여 준다. 이 두 문단의 정보를 결합하면, CPU의 코어가 제어 흐름 관리에 자원을 집중하는 구조여서 제어 기능을 최소화한 대량의 코어를 요구하는 병렬 연산 환경에 구조적으로 부적합하다는 ②

의 이유를 도출할 수 있다.

오답 해설:

- ① 스레드 블록의 불균등 분배는 GPU 내부의 문제이지 CPU가 병렬 연산 환경을 효율적으로 처리하기 어려운 이유가 아니다.
- ② 전역 메모리의 접근 속도 문제는 GPU의 메모리 구조에 관한 것으로, ②가 다루는 CPU의 한계와는 관련이 없다.
- ③ CPU의 코어당 처리량이 GPU보다 낮다는 것은 지문에 서술되어 있지 않으며, 코어의 개별 성능이 아니라 구조적 설계와 코어 수의 한계가 ②의 핵심이다.
- ④ GPU 코어의 개별 성능이 CPU보다 떨어진다는 내용은 ②가 묻고 있는 CPU의 한계에 대한 이유로 적절하지 않다.

5. 정답: ③

정답 해설: 2문단에서 ①(단순 연산 전용 코어)은 제어 기능을 최소화하고 동일한 연산을 빠르게 수행하도록 설계되어 있다고 하였다. 5문단에서 ①(분기문)이 삽입되면 스레드마다 실행해야 할 작업이 달라진다고 하였고, GPU는 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있기 때문에 병렬성이 손실된다고 하였다. 즉 ①이 삽입되어 스레드마다 다른 작업을 수행해야 하는 상황은, ①이 동일한 명령어를 동시에 수행하는 GPU의 설계 원리와 구조적으로 충돌하는 것이다.

오답 해설:

- ① ①의 수가 많아도 ②에 의한 병렬성 손실은 GPU의 설계 원리 자체에서 비롯되는 것이므로, 코어 수 증가만으로 해소되지 않는다.
- ② ①은 제어 기능이 최소화된 코어이므로, ②가 자주 등장하는 연산에서는 CPU의 고성능 코어와 동등한 효율을 발휘할 수 없다.
- ④ ①의 수를 줄이고 제어 기능을 강화하면 GPU의 구조적 특성 자체가 변질되어 연산 총량 유지가 보장되지 않으며, 이는 지문에서 다루지 않는 내용이다.
- ⑤ ①이 없는 연산에서 ①은 순차적이지 아니라 병렬적으로 명령어를 처리하며, ①이 있는 연산에서는 오히려 일부가 대기하여 병렬성이 손실된다.

6. 정답: ②

정답 해설: ④는 4문단([B])에서 레지스터와 공유 메모리를 활용한 성능 최적화 전략이 제시된 이후에 위치한다. ④ 직전에는 레지스터의 빠른 접근과 공유 메모리의 정보 공유 기능이라는 장점이 서술되어 있다. ④는 이러한 최적화 전략에도 불구하고 같은 스레드 블록 내 여러 스레드가 동시에 같은 메모리 공간에 접근할 때 충돌이 발생하여 병렬성이 저하될 수 있음을 환기함으로써, 바로 뒤에 이어지는 "메모리의 효율적 분배와 접근 방식의 조정은 GPU 연산 성능에 중요한 영향을 미친다"라는 결론을 뒷받침하는 근거로 기능한다.

오답 해설:

- ① ④에서 언급된 메모리 충돌은 공유 메모리 내부에서 발생하는 문제이지, 전역 메모리의 접근 속도 문제를 심화시키는 것이 아니다. 공유 메모리의 충돌과 전역 메모리의 접근 속도는 별개의 문제이므로 양자를 인과적으로 연결한 서술은 적절하지 않다.
- ③ ④는 충돌 시 병렬성이 저하될 수 있다고 지적할 뿐, 공유 메모리의 속도 이점 자체가 소멸된다고 서술하지 않는다. 충돌이 발생하더라도 공유 메모리가 전역 메모리보다 빠르다는 속성은 유지되므로, 공유 메모리 도입의 근거가 약화되는 것은 아니다.
- ④ ④는 전역 메모리가 연산 중에도 불가결함을 입증하지 않는다. 지문에 따르면 전역 메모리는 접근 속도가 느려 연산 중에 사용하기에 적합하지 않으며, ④의 충돌 문제가 전역 메모리의 연산 중 필요성을 입증하는 근거로 기능하지 않는다.
- ⑤ ④는 GPU 메모리 사용 시의 한 가지 주의점을 제시할 뿐, CPU와의 우열을 비교하는 맥락이 아니며, CPU의 메모리 관리 우월성이라는 결론은 지문 어디에서도 도출되지 않는다.

7. 정답: ②

정답 해설: ②는 GPU가 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있기 때문에, 분기문이 삽입되면 일부 스레드가 대기하고 병렬성이 손실된다는 내용이다. 이는 GPU가 단순 반복 연산에는 효과적이지만 복잡한 조건 판단이 요구되는 작업에서는 성능이 저하됨을 보여 준다. 6문단의 ③(연산 작업의 성격과 목적)은 CPU와 GPU 간 연산 흐름을 유기적으로 조정하는 기준인데, ②에서 드러난 GPU의 한계를 고려하면, ②은 단순 반복 연산은 GPU에, 복잡한 조건 판단 연산은 CPU에 배분하는 등 작업의 성격에 따라 적합한 장치를 선택하는 기준임을 추론할 수 있다.

오답 해설:

- ① ㉠에서 드러난 GPU의 한계는 모든 연산을 GPU에 집중시키는 설계가 최적이지 않음을 보여 주며, ㉡은 GPU 전용 작업만을 선별하는 기준이 아니라 CPU와 GPU 간 배분의 기준이다.
- ③ ㉢에서 일부 스레드가 대기하는 것은 스케줄러의 비효율이 아니라 GPU의 구조적 설계에서 비롯되는 것이며, ㉣은 스케줄러 알고리즘 개선 기준이 아니다.
- ④ ㉣에서의 병렬성 손실은 GPU의 메모리 계층 구조가 아니라 분기문에 따른 실행 경로 분리에서 비롯되며, ㉤은 메모리 접근 방식에 따른 역할 재편 기준이 아니다.
- ⑤ ㉤에서 제시된 제약은 GPU의 구조적 특성이며 CPU에 동일하게 적용되지 않는다. CPU는 복잡한 제어 흐름을 처리할 수 있도록 설계되어 있다.

8. 정답: ㉣

정답 해설: <보기>에서 윤곽 추출 과정 중 밝기 값이 기준치를 벗어나면 보정 연산을 추가 수행하고 기준치 이내이면 다음 단계로 넘어가는 조건 분기가 포함되어 있다. 5문단에서 분기문이 삽입되면 스레드마다 실행할 작업이 달라져 일부 스레드가 대기하게 되고 병렬성이 손실된다고 하였다. 따라서 윤곽 추출 단계에서의 조건 분기는 스레드 블록 내 일부 스레드의 대기를 유발하여 GPU의 병렬성 손실이 발생할 수 있으며, 해당 시스템은 이 구간에서 성능 저하를 감수해야 한다.

오답 해설:

- ① 전처리 과정에서 촬영 조건과 환자 정보를 정리하는 작업은 다양한 정보를 참조하며 복잡한 판단이 요구되는 범용 작업의 성격에 해당하므로, GPU보다는 CPU에 적합하다.
- ② 수천 장의 이미지에 동일한 필터를 적용하는 연산은 동일한 명령어를 다수의 데이터에 반복 적용하는 것이므로, CPU가 아니라 GPU의 병렬 처리 구조에 적합하다.
- ③ 밝기 값의 기준치 초과 여부에 따라 보정 연산 수행 여부가 달라지므로, 스레드마다 실행 경로가 달라져 GPU의 병렬성이 온전히 보존되지 않는다.
- ⑤ 환자별 보고서 생성 단계는 환자 이력과 촬영 기록 등 다양한 정보를 참조하며 복잡한 판단이 요구되는 작업이므로, 단순 반복 연산이 아니라 CPU에 적합한 작업이다.

9. 정답: ㉣

정답 해설: 4문단에서는 GPU의 메모리 구조를 접근 속도, 용량, 사용 범위에 따라 전역 메모리, 공유 메모리, 레지스터로 유형화하여 설명하고 있다. 그러나 전역 메모리는 접근 속도가 느려 연산 중에 사용하기에는 적합하지 않고, 연산 중에는 레지스터나 공유 메모리를 활용한다고 하였으므로, 각 메모리 유형은 고유한 용도와 한계를 가지며 상호 대체 가능한 것이 아니다.

오답 해설:

- ① 1문단은 기존 범용 작업 중심의 연산 환경과 CPU의 특성을 제시한 뒤, 대규모 반복 연산 수요 증가라는 변화 속에서 CPU의 한계를 서술하고 있으므로 적절하다.
- ② 2문단은 GPU의 코어 구조, 병렬 작동 방식 등 내부 구조를 설명한 뒤, 인공지능 학습, 영상 처리, 기상 예측 등의 활용 사례를 나열하고 있으므로 적절하다.
- ③ 3문단은 명령어 전달 → 디코딩 → 스케줄러에 의한 스레드 블록 분배 → 병렬 연산 실행 → 결과 저장의 과정을 순차적으로 서술하면서, 불균등 분배 시의 문제를 언급하고 있으므로 적절하다.
- ⑤ 5문단은 분기문과 조건 판단의 개념을 구체적 예시와 함께 설명하면서, GPU의 설계 원리와 충돌하여 병렬성이 손실될 수 있음을 서술하고 있으므로 적절하다.

10. 정답: ㉣

정답 해설:

- ㄱ: 3문단에서 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행한다고 하였다. 서로 다른 명령어를 각각 독립적으로 실행한다는 서술은 지문과 반대이므로 적절하지 않다.
- ㄴ: 2문단에서 GPU는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있다고 한 뒤, 인공지능 학습, 영상 처리, 기상 예측 등의 분야에서 동일한 연산이 반복되거나 대규모 수치 연산이 병렬로 처리되어야 한다고 하였다. 이들 분야가 동일한 연산의 반복 적용이라는 특성을 공유하기 때문에 GPU가 활용될 수 있으므로 적절하다.

ㄷ: 4문단에서 전역 메모리는 모든 스레드가 접근 가능하지만 접근 속도가 느리고, 레지스터는 접근이 매우 빠르지만 개별 스레드만 사용할 수 있다고 하였다. 접근 속도와 사용 범위 사이에서 상반된 특성을 보인다는 서술은 적절하다. 따라서 ㄴ과 ㄷ만 적절하므로 정답은 ㉣이다.

오답 해설:

- ① ㄱ만 적절하다고 보는 것은 ㄱ이 부적절하므로 오답이다.
- ② ㄴ만 적절하다고 보는 것은 ㄷ도 적절하므로 오답이다.
- ③ ㄱ과 ㄷ이 적절하다고 보는 것은 ㄱ이 부적절하므로 오답이다.
- ⑤ ㄱ, ㄴ, ㄷ 모두 적절하다고 보는 것은 ㄱ이 부적절하므로 오답이다.

11. 정답: ㉡

정답 해설: 1문단에서 ㉠(고성능 코어)은 CPU를 구성하며 복잡한 제어 흐름을 정밀하게 조율하기 위한 코어라고 하였다. 또한 ㉢(병렬 구조에 기반한 연산 장치)의 필요성은, 인공 지능 학습이나 기상 예측 등 대규모 데이터에 동일한 연산을 반복 수행해야 하는 새로운 연산 수요가 등장했는데 ㉠을 중심으로 한 CPU만으로는 이를 효율적으로 처리하기 어렵다는 한계에서 비롯된다. 따라서 ㉡가 가장 적절하다.

오답 해설:

- ① ㉠은 동일 연산의 대량 반복에 최적화된 코어가 아니라 복잡한 제어 흐름 조율에 최적화된 코어이며, ㉢은 ㉠의 구조를 계승한 것이 아니라 ㉠과는 다른 구조적 원리에 기반한 장치이다.
- ③ ㉠은 다수의 데이터에 서로 다른 명령어를 개별 적용하는 방식에 적합한 코어라는 서술은 지문에 없으며, ㉢은 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있다.
- ④ ㉠의 수를 늘려도 각 코어가 복잡한 제어 흐름 처리에 맞추어 설계되어 있으므로 GPU와 동일한 병렬 연산 효율을 달성할 수 없으며, ㉢이 임시적 대안이라는 서술도 지문에 근거가 없다.
- ⑤ ㉠은 고성능 코어이고 ㉢ 내부의 코어는 단순 연산 전용 코어이므로, 동일한 구조와 성능을 가진다는 서술은 적절하지 않다.

12. 정답: ㉤

정답 해설: 3문단의 ㉢은 연산량이 균등하게 분배되지 않으면 일부 연산 유닛이 다른 유닛의 연산이 끝날 때까지 대기 상태에 놓여 전체 GPU 자원이 비효율적으로 사용된다는 내용이다. 한편 5문단에서는 분기문이 삽입되면 스레드 블록 내 일부 스레드가 다른 스레드의 작업이 끝날 때까지 대기하게 되어 병렬성이 손실된다고 하였다. 두 현상을 비교하면, 양쪽 모두 일부 실행 단위가 다른 실행 단위의 작업 완료를 기다리며 유휴 상태에 놓인다는 점에서 구조적으로 유사하다. 그러나 ㉢은 연산 유닛 간의 연산량 불균등이라는 유닛 수준의 문제이고, 분기문에 의한 손실은 스레드 블록 내 스레드 간의 실행 경로 분리라는 스레드 수준의 문제이므로, 발생 층위가 다르다. 이 추론은 3문단과 5문단의 정보를 결합해야 도출할 수 있다.

오답 해설:

- ① ㉢의 문제는 스레드 블록 내부의 명령어 차이가 아니라 연산 유닛 간의 연산량 불균등에서 비롯되며, 분기문에 의한 병렬성 손실과는 발생 원인과 층위가 다르다.
- ② 대기 상태에 놓인 연산 유닛의 스레드 블록이 제거되어 다른 유닛으로 재분배된다는 내용은 지문에 서술되어 있지 않다.
- ③ ㉢의 문제는 분배 단계에서의 불균등에서 비롯되지만, 그 영향은 분배 이후의 병렬 연산 실행 단계에서 대기 상태라는 형태로 나타나므로, 실행 단계에서 문제가 나타나지 않는다는 서술은 적절하지 않다.
- ④ 대기 중인 연산 유닛이 추가적인 자원을 소모하지 않는다는 서술은 지문에 근거가 없다. 지문은 이러한 상황을 '전체 GPU의 자원이 비효율적으로 사용되는 문제'라고 서술하고 있으므로, 대기 중인 유닛에 할당된 하드웨어 자원이 유휴 상태로 방치되는 것 자체가 비효율에 해당한다.

13. 정답: ㉣

정답 해설: 4문단에서 공유 메모리는 같은 스레드 블록 내의 스레드들이 공동으로 사용하는 것이라고 하였다. 이는 공유 메모리의 사용 범위가 동일 스레드 블록 내부로 한정됨을 의미한다. 따라서 서로 다른 스레드 블록에 속한 스레드들 간에도 공유 메모리를 통해 데이터를 교환할 수 있다는 ㉢은 적절하지 않다.

오답 해설:

- ① 4문단에서 메모리는 접근 속도, 용량, 사용 범위 등에 따라 전역 메모리, 공유

메모리, 레지스터로 나뉜다고 하였으므로, 이를 단계적으로 배치한 구조라는 서술은 적절하다.

② 전역 메모리는 모든 스레드가 접근할 수 있는 대용량 저장 공간이지만 접근 속도가 느려 연산 중에 사용하기에는 적합하지 않다고 하였으므로 적절하다.

③ 전역 메모리의 접근 속도가 느리기 때문에 연산 중에는 레지스터나 공유 메모리를 활용한다고 하였으므로, 이들이 전역 메모리의 느린 접근 속도를 보완하기 위한 것이라는 서술은 적절하다.

⑤ 4문단 마지막에서 메모리의 효율적 분배와 접근 방식의 조정이 GPU 연산 성능에 중요한 영향을 미친다고 하였으므로 적절하다.

14. 정답: ②

정답 해설: ④는 5문단의 마지막 문장으로, GPU가 단순하고 반복적인 연산에는 효과적이지만 연산 경로가 자주 갈라지거나 복잡한 조건 판단이 요구되는 작업에서는 성능이 저하될 수 있다는 내용이다. 이는 5문단에서 전개된 분기문에 의한 병렬성 손실 논의를 종합하면서, GPU의 구조적 강점과 한계를 동시에 제시한다. 이어지는 6문단에서는 CPU와 GPU의 장점을 고려하여 두 장치의 기능을 나누어 운용하는 구조로 발전하고 있다고 서술하므로, ④는 6문단의 상호 보완적 운용이라는 결론으로 나아가기 위한 논리적 교두보로 기능한다.

오답 해설:

① ④는 GPU의 한계를 명시하고 있으므로 GPU가 자체적으로 한계를 극복할 수 있다는 논거가 될 수 없으며, 6문단은 CPU와의 협력적 운용이 필요하다는 내용 이므로 이를 불필요하다고 서술하는 것은 적절하지 않다.

③ 이후 서술인 6문단에서 분기문 최적화 기술의 발전 방향을 제시하는 내용은 없다. 6문단은 CPU와 GPU의 상호 보완적 운용을 서술하고 있다.

④ ④는 "성능이 저하될 수 있다"라고 서술할 뿐 극복 불가능하다고 단정하지 않으며, 6문단은 새로운 연산 장치의 필요성이 아니라 기존 두 장치의 협력적 운용을 서술하고 있다.

⑤ ④는 분기문에 따른 GPU의 구조적 한계를 다루는 것이지, 4문단의 메모리 충돌 문제의 해결 방안을 제시하는 것이 아니다. 5문단의 주제는 메모리 관리가 아니라 GPU의 병렬 처리 한계이다.

15. 정답: ⑤

정답 해설: 둘째 단계에서 오류가 발생한 상자를 별도의 라인으로 분리하는 것은 조건 판단에 따라 처리 경로가 달라지는 상황, 즉 분기문의 구조에 해당한다. 이는 GPU의 메모리 계층 간 데이터 이동과는 전혀 다른 원리이다. 전역 메모리에서 공유 메모리로의 데이터 이동은 접근 속도를 높이기 위한 메모리 계층 활용 전략이며, 조건에 따라 처리 경로를 분리하는 것과는 관련이 없다.

오답 해설:

① 수만 건의 상자에 동일한 공식을 반복 적용하는 연산은 동일한 명령어를 다수의 데이터에 반복 적용하는 것이므로, GPU의 병렬 처리 구조에 적합하다.

② 바코드 인식 오류 여부에 따라 처리 경로가 달라지는 상황은, 5문단에서 설명한 분기문 삽입 시 스레드마다 실행 작업이 달라져 일부 스레드가 대기하게 되는 상황과 유사한 구조이다.

③ 교통 상황, 배송 우선순위, 차량 용량 등 다양한 변수를 종합적으로 고려하는 작업은 명령어의 종류가 많고 연산 간 종속성이 큰 작업이므로, CPU의 고성능 코어가 복잡한 제어 흐름을 조율하는 데 적합하다.

④ 단순 반복 연산은 GPU에, 복잡한 판단이 필요한 연산은 CPU에 배분하는 것은 6문단에서 서술한 연산 작업의 성격과 목적에 따른 유기적 조정에 해당한다.

16. 정답: ③

정답 해설: <보기>에서 ⑥(시스템 Y)는 수천 개의 단순 연산 전용 코어를 병렬로 운용하므로, 단순 반복 연산 구간에서 빠른 처리 속도를 보인다. 그러나 5문단에서 분기문이 삽입되면 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하는 구조 때문에 일부 스레드가 대기하여 병렬성이 손실된다고 하였다. 따라서 조건 분기가 빈번한 구간에서 ⑥의 처리 속도가 저하된다. 반면 ④(시스템 X)는 소수의 고성능 코어로 복잡한 제어 흐름을 정밀하게 조율할 수 있는 구조이므로 조건 분기에 의한 성능 저하가 ⑥만큼 크지 않다. 이러한 차이로 인해 해당 구간에서 ④와 ⑥의 처리 속도 차이가 줄어든 것이다.

오답 해설:

① 단순 반복 연산 구간에서 ⑥가 빨랐던 것은 개별 코어의 성능이 높기 때문이 아니라, 수천 개의 코어가 동일한 연산을 병렬로 동시에 수행하는 구조 때문이다. 2문단에서 GPU의 코어는 제어 기능을 최소화한 단순 연산 전용 코어라고 하였

으므로, 개별 코어 성능은 오히려 ④의 고성능 코어보다 낮다.

② 조건 분기 삽입 시 ⑥의 처리 속도가 저하된 것은 전역 메모리 접근 속도가 추가로 느려지기 때문이 아니라, 스레드 블록 내에서 실행 경로가 갈라져 일부 스레드가 대기하면서 병렬성이 손실되기 때문이다.

④ ④의 장치가 전체에서 낮은 처리 속도를 보인 것은 메모리 계층 구조의 유무와 관련된 것이 아니라, 코어 수가 적어 대규모 반복 연산에서의 병렬 처리 역량이 제한되기 때문이다. 또한 지문에서 CPU가 메모리 계층 구조를 갖추고 있지 않다는 서술은 없다.

⑤ ⑥의 스케줄러가 조건 분기를 자동으로 제거한다는 내용은 지문에 서술되어 있지 않다. 스케줄러는 스레드 블록을 연산 유닛에 분배하는 역할을 수행할 뿐이다.

17. 정답: ③

정답 해설: 이 글은 1문단에서 대규모 데이터 대상 동일 연산 반복 작업의 증가라는 연산 환경 변화를 배경으로 제시하고, 2~4문단에서 GPU의 구조(코어, 연산 유닛, 메모리 계층)와 활용 분야를 설명하며, 5문단에서 분기문에 따른 병렬성 손실이라는 한계를 서술한 뒤, 6문단에서 CPU와 GPU의 상호 보완적 운용이라는 방향을 제시하고 있다. 따라서 중심 화제는 연산 환경 변화에 따른 GPU의 부상과 그 특성 및 한계이다.

오답 해설:

① 메모리 구조 개선을 위한 기술적 방안은 이 글의 중심 화제가 아니다. 메모리 구조는 GPU의 특성을 설명하는 하위 요소일 뿐이다.

② CPU의 코어 수 확장이 연산 효율에 미치는 영향은 이 글에서 다루지 않는다.

④ 병렬 연산과 순차 연산의 처리 속도에 대한 정량적 비교는 이 글에 포함되어 있지 않다.

⑤ 스레드 블록의 분배 알고리즘 최적화 전략은 이 글의 중심 화제가 아니며, 글에서는 불균등 분배의 문제를 간략히 언급할 뿐이다.

18. 정답: ④

정답 해설: 2문단에서 GPU는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있다고 한 뒤, 인공 지능 학습, 영상 처리, 기상 예측 등 다양한 분야에서 GPU가 활용되고 있다고 서술하였다. 이들 분야는 모두 대규모 데이터에 대해 동일한 연산을 반복 수행해야 한다는 공통점을 지니므로, GPU의 병렬 구조가 적합한 것이다.

오답 해설:

① CPU의 코어와 GPU의 코어가 모두 명령어를 해석하고 연산을 실행하는 것은 맞지만, CPU의 코어당 처리 데이터량이 GPU의 코어보다 적다는 서술은 지문에 없다. CPU는 고성능 코어로 복잡한 제어 흐름을 처리하고, GPU는 단순 연산 전용 코어로 동일 연산을 반복하는 것이며, 코어당 처리 데이터량의 비교는 다루지 않는다.

② 스레드 블록 내 스레드들이 동일한 명령어를 실행하는 것은 맞지만, 이는 서로 다른 연산을 동시에 처리하는 방식이 아니라 동일한 연산을 다수의 데이터에 반복 적용하는 방식에 최적화된 것이다.

③ 3문단에서 명령어가 GPU에 전달되면 디코딩 과정을 통해 해석된다고 하였으며, 이는 각 연산 유닛 내부에서 개별적으로 이루어지는 것이 아니라 GPU 차원에서 이루어진 후 스케줄러가 분배하는 구조이다.

⑤ 6문단에서 연산 작업의 성격과 목적에 따라 두 장치 간 연산 흐름을 유기적으로 조정한다고 하였으므로, CPU와 GPU의 역할이 고정되어 있다는 서술은 적절하지 않다.

19. 정답: ④

정답 해설: ②에서 GPU는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있다고 하였다. 이는 각 코어가 서로 다른 명령어를 독립적으로 해석하여 개별 데이터에 맞춤형 연산을 수행하는 것이 아니라, 모든 코어가 동일한 명령어를 각각 다른 데이터에 적용하는 것을 의미한다. 따라서 ④는 ②의 내용을 정반대로 해석한 것이므로 적절하지 않다.

오답 해설:

① 2문단에서 인공 지능 학습의 행렬 곱셈과 역전파 계산은 동일한 연산의 반복이라고 하였으므로, ②에서 서술한 최적화 방식에 부합하는 연산이라는 추론은 적절하다.

② GPU의 코어가 제어 기능을 최소화했다는 것은 복잡한 명령어 해석이나 분기 처리에 자원을 할당하지 않고, 단일 연산의 신속한 반복 실행에 자원을 집중한

것으로 추론할 수 있으므로 적절하다.

㉓ ㉔에서 모든 코어가 동일한 명령어를 수행하도록 설계되어 있다는 점은, 5문단에서 분기문 삽입 시 스레드마다 실행 작업이 달라져 병렬성이 손실되는 현상의 구조적 원인이 되므로 적절하다.

㉕ ㉖의 GPU 설계 특성(제어 기능 최소화, 동일 연산 반복)은 1문단에서 서술한 CPU의 설계 특성(고성능 코어, 복잡한 제어 흐름 조율)과 대비되며, 두 장치가 서로 다른 유형의 연산에 적합하게 설계되었음을 보여 주므로 적절하다.

20. 정답: ㉓

정답 해설: 3문단에서 명령어가 GPU에 전달되면 디코딩 과정을 통해 해석되며, ㉓(스케줄러)는 이 명령어에 따라 스레드 블록을 각 연산 유닛에 분배하고 병렬 처리가 이루어지도록 제어한다고 하였다. 따라서 ㉓은 디코딩된 명령어에 따라 스레드 블록을 연산 유닛에 분배하여 병렬 처리를 제어하는 역할을 수행한다.

오답 해설:

㉑ 3문단에서 스레드 블록 내의 스레드들은 기본적으로 동일한 명령어를 실행한다고 하였으므로, ㉑이 스레드들이 서로 다른 명령어를 번갈아 실행하도록 조율한다는 서술은 적절하지 않다.

㉒ 3문단에서 명령어의 디코딩은 GPU에 전달된 후 별도의 과정을 통해 이루어지며, ㉒은 해석된 명령어에 따라 스레드 블록을 분배하는 역할이다. ㉑이 디코딩 기능을 직접 수행하거나 명령어를 스레드 블록으로 변환한다는 서술은 지문에 근거가 없다.

㉔ 3문단에서 실행한 결과는 GPU 내의 메모리에 저장된다고만 하였을 뿐, ㉔이 결과를 수집하여 전역 메모리에 기록하거나 메모리 계층 간 데이터 이동을 관리한다는 서술은 없다.

㉕ ㉑이 분기문 삽입 시 스레드를 실행 경로별로 재분류하거나 별도의 블록으로 재구성한다는 내용은 지문에 서술되어 있지 않다.

21. 정답: ㉓

정답 해설: 4문단에서 ㉓(전역 메모리)은 모든 스레드가 접근할 수 있는 대용량 저장 공간으로 연산 실행 결과를 저장하며, 다음 단계의 연산이나 외부와의 공유를 위해 사용된다고 하였다. 다만 접근 속도가 느리기 때문에 연산 중에 사용하기에는 적합하지 않으며, 연산 중에는 레지스터나 공유 메모리를 활용한다고 하였다. 따라서 ㉓은 연산 결과 저장 및 공유의 역할을 수행하지만, 느린 접근 속도로 인해 연산 중에는 다른 메모리가 대신 활용된다는 ㉓의 추론이 적절하다.

오답 해설:

㉑ ㉓은 접근 속도가 느리고 용량이 큰 대용량 저장 공간이다. 접근 속도가 빠르고 용량이 작은 것은 레지스터의 특성이므로 적절하지 않다.

㉒ ㉓은 모든 스레드가 접근할 수 있는 저장 공간이므로, 같은 스레드 블록 내의 스레드들만 접근할 수 있다는 서술은 공유 메모리의 특성과 혼동한 것이다.

㉔ 공유 메모리를 통해 ㉓의 접근 속도 문제가 완전히 해소된다는 서술은 지문에 없으며, 4문단 마지막에서 메모리의 효율적 분배와 접근 방식의 조정이 GPU 연산 성능에 중요한 영향을 미친다고 하였으므로, ㉓의 속도와 무관하게 성능이 결정된다는 서술도 적절하지 않다.

㉕ ㉓은 개별 스레드 전용 저장소가 아니라 모든 스레드가 접근 가능한 대용량 저장 공간이다. 개별 스레드만 사용할 수 있는 저장소는 레지스터이다.

22. 정답: ㉓

정답 해설: ㉓은 4문단의 마지막 문장으로, 그 앞에서 전역 메모리의 느린 접근 속도, 레지스터와 공유 메모리의 활용을 통한 성능 최적화, 공유 메모리의 충돌 가능성 등을 차례로 서술한 뒤에 위치한다. ㉓은 이러한 논의를 종합하면서, GPU 연산 성능의 관건이 메모리의 물리적 구조 자체가 아니라 '효율적 분배와 접근 방식의 조정'이라는 운용 방식에 있음을 귀결로 제시하고 있다.

오답 해설:

㉑ ㉓은 "메모리의 효율적 분배와 접근 방식의 조정"이 중요하다고 서술하고 있으므로, 현행 메모리 계층 구조 자체를 전면 개편해야 한다는 결론이 아니라 기존 구조의 운용을 최적화해야 한다는 방향을 제시하는 것이다.

㉒ ㉓은 전역 메모리의 접근 속도 향상이 유일한 해결 방안이라고 제시하지 않는다. ㉓은 '분배'와 '접근 방식의 조정'이라는 포괄적인 방향을 제시하며, 이는 전역 메모리뿐 아니라 레지스터와 공유 메모리를 포함한 메모리 운용 전반에 해당한다.

㉔ 4문단에서 레지스터는 개별 스레드 전용이고 공유 메모리는 같은 스레드 블록 내의 스레드들이 공동으로 사용한다고 구분하였으므로, 두 유형이 기능적으로

동일하다는 전제 자체가 성립하지 않으며, ㉔가 통합을 제안하는 것도 아니다.

㉕ 3문단의 스레드 블록 불균등 분배 문제는 연산 유닛 간 연산량 배분의 문제이고, 4문단의 메모리 충돌 문제는 동일 메모리 공간에 대한 동시 접근의 문제이므로 원인이 다르다. ㉓은 4문단의 메모리 운용에 관한 귀결이며, 3문단의 문제를 함께 다루지 않는다.

23. 정답: ㉓

정답 해설: ㉓에서 '각자의 구조적 강점'이란, 글 전체의 맥락에서 CPU와 GPU 각각의 설계 특성에 근거한 강점을 가리킨다. 1문단에서 CPU는 여러 개의 고성능 코어로 구성되어 복잡한 제어 흐름을 정밀하게 조율할 수 있도록 설계되었다고 하였고, 2문단에서 GPU는 동일한 명령어를 다수의 데이터에 반복 적용하는 방식에 최적화되어 있다고 하였다. 따라서 CPU의 구조적 강점은 복잡한 제어 흐름의 조율에, GPU의 구조적 강점은 동일한 연산의 대량 병렬 처리에 해당한다.

오답 해설:

㉑ ㉓은 CPU와 GPU가 각자의 강점을 발휘하여 함께 대응한다는 내용이며, CPU의 한계가 극복되어 GPU 없이도 대응 가능하게 되었다는 내용이 아니다.

㉒ ㉓은 두 연산 장치가 각자의 강점을 발휘하는 구조를 서술하고 있으며, GPU가 CPU의 기능을 완전히 흡수하여 단일 장치로 모든 연산을 처리한다는 내용이 아니다.

㉔ '더욱 복잡하고 대규모화된 연산 환경'은 단순 반복 연산만의 증가가 아니라, 복잡한 제어 흐름이 필요한 작업과 대규모 반복 연산이 함께 요구되는 환경을 가리킨다. GPU 코어 수 확대가 핵심 전략이라는 서술도 지문에 근거가 없다.

㉕ ㉓은 두 장치 간 연산 흐름 충돌의 문제를 지적하는 것이 아니라, 두 장치 각자의 강점을 발휘하여 연산 환경에 대응하고 있다는 긍정적 귀결을 서술하고 있다.

24. 정답: ㉕

정답 해설: 4단계에서는 현재 교통 상황, 목적지까지의 경로, 법규 준수 여부 등 다양한 변수를 종합적으로 고려하여 최종 주행 판단을 내린다고 하였다. 이는 동일한 연산의 반복이 아니라 명령어의 종류가 다양하고 연산 간 종속성이 큰 복잡한 판단 작업의 성격을 지닌다. 따라서 이 작업을 GPU에 전적으로 위임하는 것이 가장 효율적이라는 서술은 적절하지 않으며, 1문단에서 서술한 CPU의 구조적 강점이 발휘되어야 하는 영역이다.

오답 해설:

㉑ 1단계에서 여러 카메라의 영상 프레임에 동일한 필터를 적용하는 작업은 동일한 명령어를 다수의 데이터에 반복 적용하는 것이므로, 2문단에서 서술한 GPU의 최적화 방식에 부합하여 적절하다.

㉒ 2단계에서 수백만 개의 픽셀 데이터에 동일한 신경망 연산을 반복 적용하는 것은, 3문단에서 서술한 GPU의 작동 원리, 즉 스케줄러가 스레드 블록을 연산 유닛에 분배하여 병렬로 연산을 실행하는 방식으로 처리될 수 있으므로 적절하다.

㉓ 3단계의 조건 분기는 5문단에서 서술한 분기문의 예시와 구조적으로 동일하며, 스레드 블록 내 스레드들의 실행 경로가 갈라져 일부 스레드가 대기하게 되는 상황을 유발할 수 있으므로 적절하다.

㉔ 3단계의 조건 분기 구간에서 발생하는 GPU의 병렬성 손실을 고려하여 해당 연산을 CPU에 위임하는 것은, 6문단에서 서술한 연산 작업의 성격과 목적에 따라 CPU와 GPU 간 연산 흐름을 유기적으로 조정하는 방식에 해당하므로 적절하다.

25. 정답: ㉓

정답 해설: 5문단에서 ㉓(조건 판단)이 자주 등장하면 스레드마다 실행할 작업이 달라지고, GPU는 스레드 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있기 때문에 일부 스레드가 대기하여 병렬성이 손실된다고 하였다. <보기>에서 각 유닛의 블록에는 두 개의 스레드가 있고 유닛 내 스레드의 수와 연산량은 비례한다고 하였는데, 이 조건은 모든 스레드가 동일한 연산을 정상적으로 수행할 때 성립하는 것이다. ㉓이 개입하여 두 스레드의 실행 경로가 갈라지면 한 스레드는 다른 스레드의 작업이 끝날 때까지 대기하게 되므로, 스레드 수에 비례하는 연산량이 확보되지 않아 실질 처리 효율이 저하된다. 이 추론은 3문단의 GPU 구조와 5문단의 분기문 효과를 <보기>의 조건에 결합해야 도출할 수 있다.

오답 해설:

㉑ 스케줄러가 블록을 해체하고 동일 경로의 스레드끼리 재 묶어 재분배한다는 내

용은 지문에 서술되어 있지 않다. 3문단에서 스케줄러는 스레드 블록을 연산 유닛에 분배하는 역할을 수행할 뿐이다.

② <보기>에서 모든 스레드가 동일한 성능으로 작동한다는 것은 개별 스레드의 연산 처리 능력이 동일하다는 의미이지, 서로 다른 실행 경로를 독립적으로 동시 수행할 수 있다는 의미가 아니다. 5문단에 따르면 GPU는 블록 내 모든 스레드가 동일한 명령어를 동시에 수행하도록 설계되어 있으므로, 실행 경로가 같아지면 일부 스레드가 대기하게 되어 병렬성이 손실된다.

④ ◎으로 인한 성능 저하는 하단의 메모리(전역 메모리) 접근 과정에서의 충돌이 아니라, 스레드 블록 내에서 실행 경로가 달라져 일부 스레드가 대기하는 데서 비롯된다. 메모리 충돌은 4문단에서 다른 별개의 문제이다.

⑤ ◎으로 인한 병렬성 손실은 스레드 블록 내부에서 스레드 간 실행 경로가 달라지는 데서 비롯되는 것이지, 유닛 간 연산량 불균등 분배와는 발생 층위가 다르다. 전자는 블록 내 스레드 수준의 문제이고 후자는 유닛 수준의 문제이므로, 스케줄러의 분배 기능 개선으로 해소될 수 있는 것이 아니다.